INDUSTRIAL PROJECT REPORT

(Internship Semester Jan–June 2026)

"AI-POWERED PROCTORING SYSTEM FOR ONLINE EXAMINATION INTEGRITY"

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

ASHIT VIJAY

Student Regn. No. 22BCON1528

Algorizz Technologies Pvt. Ltd.

HSR Layout, Bengaluru, Karnataka – 560102

Under the Guidance of

Faculty Internship Guide
Name : Mr. Mahesh Joshi
Designation : Assistant Professor

Industry Guide
Name : Mr. Rajneesh Mittal
Designation : Founder & CEO

Department of Computer Science and Engineering

JECRC UNIVERSITY, JAIPUR

July 2026

DECLARATION

I hereby declare that the project work entitled "AI-Powered Proctoring System for Online Examination Integrity" is an authentic record of my own work carried out at Algorizz Technologies Private Limited, Bengaluru, as a requirement of the six-month industrial project for the award of the degree of B.Tech (Computer Science and Engineering) from JECRC University, Jaipur, under the guidance of Mr. Rajneesh Mittal (Industry Guide) and Mr. Mahesh Joshi (Faculty Internship Guide), during January to June 2026.

The matter embodied in this report has not been submitted for the award of any other degree or diploma. All references and sources of information have been properly acknowledged.

Signature of Student

ASHIT VIJAY

Reg. No. 22BCON1528

Date: _____

It is certified that the above statement made by the student is correct to the best of our knowledge.

Signature
Mr. Mahesh Joshi
Assistant Professor, CSE
Faculty Internship Guide

Signature
Mr. Rajneesh Mittal
Founder & CEO, Algorizz Technologies
Industry Guide

Signature
Dr. Naveen Hemrajani
Dean – Engineering

Signature
Dr. Bhavana Sharma
HOD – CSE

CERTIFICATE

JECRC University, Jaipur

Department of Computer Science and Engineering

This is to certify that the project entitled "AI-Powered Proctoring System for Online Examination Integrity" has been carried out by Ashit Vijay (Reg. No. 22BCON1528) at Algorizz Technologies Private Limited, Bengaluru, during the period January to June 2026, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from JECRC University, Jaipur.

The work has been carried out under our supervision and guidance and has not been submitted elsewhere for any other degree or diploma. The student has fulfilled all requirements and the report is satisfactory.

Signature: _____

Mr. Mahesh Joshi
Assistant Professor, CSE
Faculty Internship Guide
JECRC University, Jaipur

Signature: _____

Mr. Rajneesh Mittal
Founder & CEO
Industry Guide
Algorizz Technologies Pvt. Ltd.

Seal of Company:

[Company Seal / Stamp]

Dr. Naveen Hemrajani
Dean – Engineering

Dr. Bhavana Sharma
HOD – CSE

PREFACE

This report documents the complete six-month industrial internship (January–June 2026) undertaken by Ashit Vijay (22BCON1528) at Algorizz Technologies Private Limited, Bengaluru, as part of the B.Tech Computer Science and Engineering program at JECRC University, Jaipur.

The internship project — "AI-Powered Proctoring System for Online Examination Integrity" — is a full-stack AI/ML backend system for real-time remote exam monitoring. The system was built from scratch over seven development phases, progressing from a basic REST API scaffold to a production-ready platform integrating Computer Vision (MediaPipe, YOLOv11), Audio Signal Processing (Silero VAD, librosa, torchaudio), concurrent multi-threaded processing, and cloud-based persistence (PostgreSQL, Redis).

The report is organized according to the prescribed JECRC University End-Semester Internship Report format. It covers the project background, company profile, methodology, system design, implementation details, testing results, findings, conclusions, recommendations, and learning outcomes. Annexures include screenshots of the developed system, API responses, database records, and sample proctoring session reports.

This report is intended as both a technical documentation of the work performed and a reflective account of the professional and academic growth experienced during the internship.

The structure of the report follows the prescribed JECRC University End-Semester format closely, with chapters covering Introduction, Company Profile, Methodology, Extended Technical Analysis, Discussion and Findings, Conclusion, Recommendations and Learning, and Appendices. The appendices include actual API response samples, database

schema documentation, sample session reports, system configuration details, a comprehensive weekly work log, and a technical glossary.

A note on terminology: throughout this report, the terms 'candidate' and 'examinee' refer to the student or professional taking the online examination being monitored by the proctoring system. The term 'invigilator' or 'proctor' refers to the human reviewer who examines flagged sessions post-exam. The term 'confidence score' refers to the AI model's estimated probability that a detected event is a genuine positive (range 0.0 to 1.0), not to a statistical confidence interval in the hypothesis testing sense.

All technical measurements, performance benchmarks, and accuracy figures reported in this document were obtained from tests conducted on the development hardware described in Section 3.5. Performance on different hardware configurations (particularly GPU-enabled servers) would differ substantially from the CPU-only benchmarks presented here.

Ashit Vijay

22BCON1528

B.Tech CSE, JECRC University, Jaipur

July 2026

ACKNOWLEDGEMENTS

The completion of this industrial internship and the preparation of this report would not have been possible without the support, guidance, and encouragement of numerous individuals, and it is a pleasure to express my gratitude to all of them.

First and foremost, I express my deepest gratitude to Mr. Rajneesh Mittal, Founder and CEO of Algorizz Technologies Private Limited, who served as my Industry Guide throughout this internship. His mentorship, technical insight, and entrepreneurial perspective have profoundly influenced my understanding of AI product development. His willingness to assign challenging, production-grade work rather than routine tasks gave me the rare opportunity to build a real system that addresses a real-world problem.

I am sincerely grateful to my Faculty Internship Guide, Mr. Mahesh Joshi, Assistant Professor, Department of Computer Science and Engineering, JECRC University, Jaipur. His academic guidance, periodic reviews, and encouragement throughout the six months ensured that the project remained aligned with both industry requirements and academic standards.

I extend my heartfelt thanks to Dr. Bhavana Sharma, Head of Department, CSE, JECRC University, for her leadership and support throughout the internship program. I am also grateful to Dr. Naveen Hemrajani, Dean of Engineering, for creating an institutional environment that values industry engagement and practical learning.

I would like to thank the entire team at Algorizz Technologies, particularly the engineering and product teams, whose collaborative spirit and technical expertise created an ideal learning environment. The experience of working in a fast-paced AI startup — where every

team member's contribution has immediate and tangible impact — has been immensely valuable.

I am also thankful to the open-source communities behind FastAPI, MediaPipe, Ultralytics YOLO, Silero VAD, librosa, torchaudio, SQLAlchemy, and Redis. Their well-documented, production-quality libraries made it possible to build a sophisticated system within the constraints of a six-month internship.

Finally, I thank my family and friends for their constant encouragement, patience, and support throughout this journey.

Ashit Vijay

22BCON1528

TABLE OF CONTENTS

LIST OF ILLUSTRATIONS

ABSTRACT / EXECUTIVE SUMMARY

This report presents the complete work undertaken during a six-month industrial internship (January–June 2026) at Algorizz Technologies Private Limited, Bengaluru, by Ashit Vijay (22BCON1528), B.Tech Computer Science and Engineering, JECRC University, Jaipur.

The internship project — "AI-Powered Proctoring System for Online Examination Integrity" — is a production-grade backend platform designed to monitor online examinations in real time. The system accepts live video frames and audio chunks from a candidate's browser during an examination and applies a battery of AI and signal processing algorithms to detect suspicious behaviour, compute a risk score, and generate detailed post-exam reports for review by invigilators.

The system integrates three categories of detection: (1) Video-based detection using MediaPipe FaceDetection (face count), MediaPipe FaceMesh with OpenCV solvePnP (3D head pose estimation), and YOLOv11 (forbidden object detection — phones, laptops, books, tablets); (2) Audio-based detection using Silero VAD (voice activity), librosa (MFCC, spectral features, pitch), and scipy (Welch PSD) for six independent audio analyzers covering speaker count, whispering, speech rhythm patterns, loud noise, and background media; and (3) A weighted risk scoring engine that produces a normalized 0–100 risk score per session.

The backend is built with FastAPI (Python), persists data to PostgreSQL (Neon serverless) via SQLAlchemy ORM, and uses Redis for real-time temporal state management (pose smoothing, object streak tracking). A critical engineering challenge — a malloc heap corruption crash caused by concurrent native library calls — was identified and resolved through systematic log analysis and correct asyncio executor dispatch design.

The development followed a structured 7-phase model over the six months, culminating in a fully tested, deployed system. Key outcomes include: 9 independent detection modules, 6 bug fixes (FIX 1–6) covering thread safety and algorithm correctness, reduction of false positive rates to below 5% for all critical detectors, and a deployment configuration supporting concurrent multi-session processing.

Keywords:

AI Proctoring, Online Examination Integrity, FastAPI, Computer Vision, MediaPipe, YOLOv11, Voice Activity Detection, Silero VAD, MFCC, Head Pose Estimation, Thread Safety, Python Concurrency, PostgreSQL, Redis, Machine Learning, Audio Signal Processing.

Technical Innovation Summary

The system introduces several engineering innovations beyond straightforward model integration. The adaptive camera calibration system — which infers focal length from the candidate's face size rather than requiring explicit camera calibration — is a practical solution to a real deployment challenge: online exam candidates use a wide variety of devices with different sensor sizes, focal lengths, and field-of-view characteristics. By normalizing pose estimation to the observed face geometry, the system achieves consistent pose accuracy across device types without requiring any candidate action.

The multi-gate false positive architecture — requiring multiple independent acoustic or visual conditions to be simultaneously satisfied before flagging an event — represents a departure from conventional single-threshold detection. This design philosophy, applied consistently across all six audio detectors and the head pose detector, was the primary driver of reducing false positive rates from 15–65% (pre-tuning) to below 5% (post-tuning). The approach is generalizable to any behavioral monitoring system where false accusations carry significant consequences.

The thread-local ML model architecture — where each of the six frame processing threads maintains its own independent instances of MediaPipe FaceDetection, FaceMesh, and YOLO — provides safe concurrent inference without the synchronization overhead of mutex-protected shared models. This design achieves near-linear scaling with the number of frame workers (6 workers processing 6 frames simultaneously in ~200ms, vs. a single worker taking 1200ms sequentially) while eliminating all inter-thread interference.

CHAPTER 1: INTRODUCTION

The proliferation of digital technology and the unprecedented disruption caused by the COVID-19 pandemic have fundamentally altered how academic examinations, professional certifications, and corporate assessments are conducted globally. Universities, schools, professional bodies, and employers alike have shifted to online platforms, making remote proctoring not a convenience but a necessity. According to a 2024 report by Grand View Research, the global online proctoring market was valued at USD 747 million in 2023 and is projected to grow at a CAGR of 17.2% through 2030, driven by the exponential growth of online education, MOOCs, and remote hiring.

However, the transition to remote examination has introduced significant integrity challenges. Without physical supervision, candidates have far greater opportunity to cheat through unauthorized reference materials, assistance from other individuals, secondary devices, and coordinated communication. Human online proctors — individuals watching live video feeds — are expensive (estimated USD 10–30 per exam hour), prone to attention fatigue, and unable to process audio-visual signals as consistently and objectively as an automated system. There is, therefore, a strong and growing demand for automated AI-based proctoring solutions that can monitor multiple candidates simultaneously, detect suspicious behavior consistently, and provide objective audit trails.

1.1 Background and Motivation

Algorizz Technologies identified a market gap in the proctoring space: most commercial solutions (ProctorU, Examity, Honorlock) are cloud-only SaaS products with high per-exam pricing (USD 5–30 per candidate), opaque AI models, and poor support for low-bandwidth environments. Algorizz's vision was to build a lightweight, open-architecture, API-first proctoring backend that educational institutions could self-host, customize, and integrate with their existing LMS platforms.

As the AI/ML Developer Intern assigned to this project, my role was to design and build the complete backend system from the ground up — from the FastAPI REST API layer through the AI processing pipeline to the database persistence layer. This project provided the opportunity to apply academic knowledge in Computer Vision, Digital Signal Processing, and Software Engineering in a production context with real quality and reliability requirements.

1.2 Problem Statement

The fundamental problem addressed by this project is: how can an automated AI system reliably and fairly monitor a candidate during an online examination, detecting genuine cheating behavior while minimizing false accusations of honest candidates?

This decomposes into several specific technical challenges:

- Video monitoring: detecting when a candidate looks away from the screen, when unauthorized persons enter the frame, or when forbidden physical objects (phones, books, additional screens) appear in the camera view.
- Audio monitoring: detecting when a second person is speaking in the room, when the candidate is whispering (potentially communicating with an accomplice), when speech follows a suspicious pattern (reading answers aloud), or when background media (TV, radio) is playing.
- Concurrent processing: handling multiple exam sessions simultaneously without cross-session interference or performance degradation.
- False positive minimization: ensuring honest candidates are not flagged for normal exam behaviors (looking slightly away while thinking, natural speech variation, background ambient sounds).

- Real-time operation: processing video at 2–3 frames per second and audio every 5 seconds with end-to-end latency below 500ms per chunk.
- Auditability: storing all detected events with confidence scores, timestamps, and metadata to enable post-exam human review.

1.3 Objectives

The primary objectives of this internship project are:

1. Design and implement a production-grade REST API using FastAPI (Python) with full asynchronous request handling, Pydantic data validation, and complete session lifecycle management.
2. Build a thread-safe concurrent processing architecture supporting 6 parallel video frame workers and 1 serialized audio worker using Python ThreadPoolExecutors and asyncio.
3. Integrate MediaPipe FaceDetection and FaceMesh for face count monitoring and 3D head pose estimation (yaw, pitch, roll) using the OpenCV solvePnP algorithm with adaptive camera calibration.
4. Integrate YOLOv11 (Ultralytics) for real-time forbidden object detection with per-class confidence thresholds, edge-aware detection, temporal IoU tracking, and size-based filtering.
5. Build a 6-module audio analysis pipeline covering Voice Activity Detection, multi-speaker identification, whisper detection, speech rhythm pattern analysis, loud noise detection, and background media detection.
6. Design a weighted risk scoring engine producing normalized 0–100 session risk scores with automated Low/Medium/High risk classification and FLAGGED/COMPLETED session status assignment.
7. Implement persistent session and event storage using PostgreSQL (Neon serverless) via SQLAlchemy ORM with connection pooling, and real-time state management using Redis.

8. Identify and resolve critical engineering issues including a malloc heap corruption, MediaPipe thread safety, YOLO initialization race conditions, and audio detector algorithm correctness bugs.
9. Conduct comprehensive testing including unit testing, concurrency stress testing, false positive analysis, integration testing, and performance benchmarking.

1.4 Scope and Limitations

1.4.1 Scope

The following components are within the scope of this internship project:

- Complete FastAPI backend with 12 REST API endpoints covering session management, media ingestion, status monitoring, configuration, debug analysis, and report generation.
- Video processing pipeline: base64 image decoding, MediaPipe face and mesh detection, OpenCV-based 3D head pose estimation, Redis-backed pose smoothing, YOLOv11 object detection with temporal tracking.
- Audio processing pipeline: multi-format WebM/Opus decoding via FFmpeg and torchaudio, 16kHz resampling, Silero VAD, and 5 downstream audio feature analyzers.
- PostgreSQL database schema design, SQLAlchemy ORM model implementation, and connection pool configuration.
- Redis integration for temporal state (pose history, object tracking streaks, session counters, risk score caching).
- Adaptive camera calibration system with 5 presets and dynamic focal length estimation.
- Weighted risk scoring engine with 16 event type weights.
- Debug and configuration endpoints for frontend calibration and runtime threshold adjustment.

- Deployment configuration for Uvicorn ASGI server and cloud infrastructure (Neon PostgreSQL).

1.4.2 Limitations

The following are outside the scope of this internship and represent known limitations of the current system:

- No frontend browser application — the system is a backend API only. The JavaScript MediaRecorder and getUserMedia capture pipeline must be provided by an external frontend.
- No GPU acceleration — all inference runs on CPU. YOLO inference at 2–3 FPS on CPU is adequate, but higher frame rates would require CUDA-enabled deployment.
- No formal speaker diarization model — speaker count estimation uses MFCC variance, which is less accurate than dedicated diarization (e.g., pyannote.audio) for complex scenarios.
- No gaze estimation — head pose is used as a proxy for gaze direction, which is a valid but imperfect approximation. Iris-based gaze tracking would be more precise.
- No user authentication or exam scheduling — the API assumes an external system manages candidate identities and exam assignments.
- Single-process deployment — horizontal scaling requires multiple server instances behind a load balancer rather than Uvicorn multi-worker mode.

CHAPTER 2: COMPANY PROFILE — ALGORIZZ TECHNOLOGIES

Algorizz Technologies Private Limited is an emerging Artificial Intelligence and Digital Transformation company headquartered in Bengaluru, Karnataka, India. Incorporated on December 15, 2023, under the Ministry of Corporate Affairs, Government of India (CIN: U63990KA2023PTC182303), the company has rapidly established itself as a credible partner for businesses seeking to harness the power of AI, data analytics, and modern software engineering.

2.1 Company Identification and Registration Details

Table 2.1: Algorizz Technologies — Company Profile

Attribute	Details
Registered Company Name	Algorizz Technologies Private Limited
CIN	U63990KA2023PTC182303
Date of Incorporation	December 15, 2023
Type of Company	Private Limited Company (Pvt. Ltd.) — Section 2(68), Companies Act 2013
Registrar of Companies	ROC Bangalore, Karnataka
Registered Address	1908, Tower 4, Shriram Chirpingwoods, 12th Main, HSR Layout, Bengaluru, Karnataka – 560102
Founder and CEO	Mr. Rajneesh Mittal
Director	Ms. Soni Jalan
Industry Classification	Information Technology / AI / Data Analytics / Software Development
Website	www.algorizz.com

Team Size	20–25 (Early-stage growth startup)
Revenue (FY 2024–25)	₹35.8 Lakhs (616% Year-on-Year CAGR)
Primary Markets	India (primary), APAC, Europe, United States
Engagement Models	Fixed-Price Projects | Retainer Partnerships | Staff Augmentation

2.2 Founding Vision and Leadership

Algorizz Technologies was founded by Rajneesh Mittal, a technology leader with 25 years of experience across marquee organizations in India and internationally. Mr. Mittal has served in senior technology roles at Vodafone, Hughes Network Systems, Reliance Industries, Zee Entertainment Enterprises, Hewlett Packard (HP), and Manipal Group. His career has spanned telecommunications, media, enterprise IT, and education — giving him a uniquely broad perspective on how technology creates business value across diverse industries.

Having served as a Chief Technology Officer (CTO) and Chief Information Officer (CIO) for multiple large enterprises, Mr. Mittal recognized a critical gap in the market: while large corporations can afford dedicated AI teams and big-budget digital transformation programs, Small and Medium Businesses (SMBs), startups, and mid-sized enterprises are largely excluded from these benefits due to the high cost and complexity of enterprise AI adoption. Algorizz Technologies was founded to address this gap — making production-quality AI and digital capabilities accessible to organizations of all sizes.

The company's founding philosophy is captured in four strategic principles:

- Build Fast, Scale Faster: Phased rollouts that deliver immediate business value and expand seamlessly as the client's needs grow — prioritizing working software over comprehensive upfront planning.

- Agile Innovation, Instant Impact: Leveraging proprietary technology frameworks and pre-built AI accelerators to help clients launch faster and iterate quickly based on real-world feedback.
- Your Tech, Unbiased: Vendor-neutral technology recommendations driven entirely by client goals and technical fit — never by vendor partnerships or commissions. Clients receive the right technology for their context, not the most profitable recommendation.
- Essential Tech, Maximum Impact: Lean, focused solutions that solve specific problems efficiently, avoiding over-engineering and scope creep that inflate costs without proportional business benefit.

2.3 The Algorizz Product and Service Stack

Algorizz Technologies organizes its offerings across four integrated service lines that together form the complete "Algorizz Stack" — a comprehensive capability set covering strategy, software, data, and AI:

2.3.1 Consultancy

The consultancy pillar provides strategic technology leadership to organizations that need CTO-level thinking without the cost of a full-time executive. Services include:

- Virtual CTO/CIO Services: Fractional technology leadership for startups and SMBs — reviewing architecture decisions, evaluating technology vendors, designing engineering hiring plans, and representing technology strategy in board discussions.
- Tech.Data.AI Bootcamp: Intensive training programs for leadership teams and engineers to build internal AI and data literacy, reducing dependency on external consultants for technology decisions.

- Cohesive Ecosystem Integration: Audit and rationalization of a client's technology stack — eliminating redundant tools, integrating fragmented systems, and establishing a unified data and communication infrastructure.
- AI Infused Roadmaps: Pragmatic, prioritized AI adoption strategies that identify high-ROI use cases for AI within a client's specific business context, with phased implementation plans.

2.3.2 Software Development

The software development pillar delivers complete product engineering services across the full product lifecycle:

- Modern UX/UI Design: User-centered design processes (personas, journey mapping, wireframing, prototyping) that prioritize adoption and reduce training requirements.
- Scalable Tech Platforms: Architecture design and implementation for high-availability, horizontally scalable backend systems using microservices, event-driven architecture, and cloud-native patterns.
- Enterprise Systems: Custom ERP, CRM, HRMS, and workflow automation systems tailored to specific industry requirements, delivered at a fraction of the cost of commercial off-the-shelf enterprise software.
- Product Lifecycle Management: End-to-end product development from concept → MVP → growth → scale, with Agile project management and continuous delivery practices.

2.3.3 Data and Analytics

The data and analytics pillar is anchored by Algorizz's flagship product, qRIZZ, and supported by a comprehensive data engineering services portfolio:

- qRIZZ — GenAI-Powered Analytics: Algorizz's proprietary product that connects to multiple data sources (databases, spreadsheets, APIs, documents) and enables business users to query their data in natural language, receiving AI-generated analysis and visualizations without writing SQL or code.
- Data Foundations: Building the fundamental data infrastructure for analytics — data lakes (raw storage), data warehouses (structured analytics), ETL pipelines, and data quality frameworks.
- Decision Dashboards: Real-time customizable business intelligence dashboards built on top of the data foundation, providing leadership teams with single-screen visibility into key business metrics.
- Language Driven Insights: Natural language interfaces on top of structured enterprise data, enabling non-technical business stakeholders to ask questions of their data in plain English.

2.3.4 Artificial Intelligence

The AI pillar represents Algorizz's most advanced technical capabilities, directly relevant to the internship project:

- AI-Powered Vision: Computer Vision solutions for automated visual inspection, quality control, safety monitoring, and behavioral analysis. The AI Proctoring System built during this internship is a flagship example of this capability.
- Predictive Intelligence: Machine Learning models for business forecasting, demand prediction, churn prediction, risk scoring, and recommendation systems.
- GenAI Accelerators: Industry-specific conversational AI products including document analyzers, knowledge base chatbots, code generation tools, and content automation systems.
- Agentic AI Systems: Autonomous AI agents that observe, plan, and act to complete multi-step business workflows without human intervention — the most advanced tier of Algorizz's AI offerings.

2.4 Industry Context and Market Opportunity

Algorizz Technologies operates at the intersection of several high-growth technology markets:

Table 2.2: Market Segment Analysis and Growth Opportunity

Market Segment	2023 Market Size	2030 Projected	CAGR	Source
AI in Education	USD 4.0 Billion	USD 30.0 Billion	32.9%	Grand View Research
Online Proctoring	USD 747 Million	USD 2.5 Billion	17.2%	Grand View Research
Computer Vision	USD 14.9 Billion	USD 48.6 Billion	18.4%	MarketsandMarkets
AI SaaS (India)	USD 1.2 Billion	USD 9.8 Billion	34.5%	NASSCOM 2024
Generative AI	USD 28.0 Billion	USD 356 Billion	44.3%	Bloomberg Intelligence

India's positioning as a global AI talent hub — with the third-largest pool of AI researchers globally and a rapidly growing base of AI startups — provides Algorizz Technologies with both a talent advantage and a large domestic market opportunity. The Indian government's focus on AI adoption through initiatives like the India AI Mission (₹10,371 crore budget over five years) further strengthens the addressable market for AI services companies.

2.5 The Internship Experience at Algorizz Technologies

The internship at Algorizz Technologies provided an experience qualitatively different from internships at large established technology companies (Infosys, Wipro, TCS, etc.) in several important ways:

2.5.1 Ownership and Impact

At a startup with a 20–25 person team, every engineer's contribution is directly visible in the product. The AI Proctoring System built during this internship was not a training exercise or a minor feature on an existing product — it was a greenfield production system that the company plans to take to market. This level of ownership and accountability provided motivation and learning opportunities that would be impossible in a large organization where interns typically work on isolated sub-components of massive systems.

2.5.2 Technical Breadth

The project required contributions across the full technology stack: API design, AI model integration, database engineering, caching strategy, concurrent programming, testing, and deployment configuration. This breadth of scope in a single internship project is characteristic of startup engineering culture, where generalist engineers who can contribute across the stack are highly valued over narrow specialists.

2.5.3 Direct Mentorship

Working directly with the Founder and CEO as the industry guide provided access to mentorship at a level that is rarely available in large organizations. Discussions about technology strategy, product-market fit, client requirements, and engineering trade-offs with an experienced CTO-level mentor added a strategic dimension to the technical learning that significantly enriched the internship experience.

2.5.4 Startup Operations

Observing the operations of an early-stage AI startup — including business development conversations, client requirement analysis, technology stack decision-making, resource prioritization, and rapid iteration cycles — provided valuable exposure to the entrepreneurial aspects of technology that are not covered in academic curricula.

2.6 Work Environment and Culture

Algorizz Technologies operates with a remote-first culture, with team members distributed across Bengaluru, Jaipur, and other cities. Communication is primarily asynchronous (Slack, email, documentation) with regular synchronous touchpoints (weekly standup, bi-weekly technical review, monthly all-hands). This remote-first environment required developing strong written communication skills and the discipline of asynchronous work — practices that are increasingly important in the modern software industry.

The company's engineering culture emphasizes documentation, code quality, and structured problem-solving. The practice of documenting all bug fixes with numbered entries (FIX 1 through FIX 6) in the source code header, and maintaining phase-by-phase development notes, reflects a culture that values institutional knowledge preservation — a mature engineering practice that is often absent in early-stage startups but was deliberately cultivated at Algorizz.

CHAPTER 2: METHODOLOGY

This chapter describes the methods, procedures, tools, and phased development approach used to design and build the AI Proctoring System. The development followed an iterative, phase-based methodology inspired by Agile software development principles, with each phase delivering a working, testable increment of the system.

2.1 Project Development Approach — 7-Phase Model

Rather than a traditional waterfall approach where all design is completed before implementation begins, the project used a phased iterative model where each phase introduced a new set of detection capabilities, tested them independently, and integrated them with the existing system before moving to the next phase. This approach minimized integration risk and ensured that each new module could be tested in isolation before becoming part of the live processing pipeline.

Table 2.1: Phase-wise Development Summary

Phase	Month	Focus Area	Key Deliverables
1	January 2026	Setup & Architecture	FastAPI scaffold, PostgreSQL schema, Redis integration, session lifecycle APIs (/create, /end, /status), health endpoint, CORS configuration, environment setup.
2	February 2026	Face Detection	MediaPipe FaceDetection integration, NO_FACE and MULTIPLE_FACES event generation, base64 image decoding pipeline, cv2.flip mirror correction, /frames endpoint.
3	March 2026	Head Pose Estimation	MediaPipe FaceMesh, 6-point solvePnP pose estimation, adaptive camera calibration (5 presets), Redis-backed 3-frame pose smoothing, temporal streak logic (2-frame), LOOKING_AWAY events.

4	April 2026	Audio Pipeline	Silero VAD integration, 4-attempt audio format fallback (webm/ogg/mp4/auto), BACKGROUND_SPEECH detection, MULTIPLE_SPEAKERS (MFCC+pitch variance), WHISPERING (RMS+spectral flatness), SPEECH_PATTERNS (reading/Q&A rhythm), LOUD_NOISE, BACKGROUND_MEDIA (Welch PSD). /audio endpoint.
5	May 2026	Object Detection	YOLOv8n integration (later upgraded to YOLOv11n), EnhancedObjectDetector class, DETECTION_CONFIG per-class thresholds, edge detection, size validation (width/area ratios), Redis temporal IoU tracking (5-frame history), deterministic torch.manual_seed(42).
6	May–Jun 2026	Thread Safety	FIX 1: run_in_executor (malloc crash fix). FIX 2: thread-local MediaPipe. FIX 3: thread-local YOLO + _yolo_init_lock. FIX 4: YOLOv8→YOLOv11. FIX 5: whisper detection correction. FIX 6: -50dB energy gate. Risk scoring engine finalization.
7	June 2026	Testing & Deploy	Unit testing (all 9 detectors), concurrency stress test (500 concurrent requests, 0 crashes), false positive analysis & threshold tuning, integration testing (full lifecycle), performance benchmarking, Neon PostgreSQL deployment, /debug/analyze-frame endpoint.

2.2 Phase 1 — Project Setup and Architecture Design

The first phase established the foundational architecture of the system before any AI models were integrated. This phase focused on getting the basic plumbing right: API structure, database schema, and inter-service connectivity.

2.2.1 Technology Stack Selection

FastAPI was chosen as the web framework over Flask and Django for three reasons: (1) native async/await support for non-blocking I/O operations, (2) automatic OpenAPI documentation generation from Pydantic models, and (3) Starlette's high-performance

ASGI request handling. Python 3.11 was selected for its improved performance (10–60% faster than Python 3.9 in benchmarks) and improved asyncio error messages.

PostgreSQL was chosen as the primary database for its JSONB column support (used for event metadata storage without a rigid schema), ACID compliance (critical for audit log integrity), and availability as a serverless offering through Neon (which provides automatic scaling and connection pooling at the infrastructure level).

Redis was chosen for temporal state management because its native TTL (time-to-live) support on keys perfectly matches the requirement for pose history (300s TTL) and object tracking history (10s TTL) that automatically expire after their relevant window.

2.2.2 Database Schema Design

Two primary database tables were designed:

The SessionDB table captures session-level information: session ID (UUID primary key), candidate_id, exam_id, status (ACTIVE/COMPLETED/FLAGGED), risk_score (float), started_at and ended_at timestamps, exam_duration_minutes, and a JSONB metadata_ column for candidate name and email.

The EventDB table captures individual detection events: event ID (UUID primary key), session_id (indexed foreign key for fast per-session queries), event_type (string enum), severity (LOW/MEDIUM/HIGH/CRITICAL), confidence_score (float 0–1), model_version (string), timestamp, and a JSONB metadata_ column for event-specific data (bounding boxes, dB levels, spectral features, pose angles).

2.2.3 API Design Principles

The API was designed to be asynchronous and non-blocking. All compute-intensive operations (AI model inference) are dispatched to background thread pools so that the HTTP response returns immediately (HTTP 202 Accepted) without waiting for processing to complete. This ensures the API can handle high request rates even when individual frames take 200–300ms to process.

2.3 Phase 2 — Face Detection Module

The face detection module uses MediaPipe FaceDetection (model_selection=1 for range > 2 metres, min_detection_confidence=0.7) to count the number of faces in each video frame. Face count anomalies are the most straightforward indicators of exam integrity violations.

Table 2.2: Face Detection Event Types

Event Type	Condition	Severity	Confidence	Interpretation
NO_FACE	num_faces == 0	HIGH	0.95	Candidate has left frame or is obscured
MULTIPLE_FACES	num_faces > 1	CRITICAL	0.90	Another person present during exam

A critical implementation detail discovered during Phase 2 was the need for cv2.flip(frame, 1) before MediaPipe processing. The browser's getUserMedia API, when used without the mirror: false constraint on the video track, sends a horizontally mirrored stream. Without the flip correction, left and right eye positions are swapped, causing yaw detection to be inverted in Phase 3. The flip was applied to the decoded frame before any MediaPipe or YOLO processing.

2.4 Phase 3 — Head Pose Estimation Module

Head pose estimation determines whether a candidate's head is oriented toward the screen (normal exam behaviour) or directed away (suspicious). The system uses a 6-point facial landmark approach combined with OpenCV's Perspective-n-Point (PnP) algorithm for 3D pose recovery.

2.4.1 MediaPipe FaceMesh Configuration

FaceMesh is configured with max_num_faces=2 (to detect mesh on up to 2 faces, though pose is computed only for the primary face when exactly 1 is detected), refine_landmarks=True (enables iris landmark detection for higher-precision outer eye corners), min_detection_confidence=0.7, and min_tracking_confidence=0.7.

2.4.2 Six-Point Landmark Model

Six anatomically stable facial landmarks are extracted from the 478-point FaceMesh for the PnP computation:

Table 2.3: Head Pose Estimation Landmark Points

MediaPipe Index	Facial Location	3D Model Coordinates (mm)	Role in PnP
1	Nose tip	[0.0, 0.0, 0.0]	Origin — primary reference point
152	Chin	[0.0, -6.5, -2.5]	Vertical axis stability
33	Left eye outer corner	[-4.0, 3.5, -1.5]	Horizontal axis (left)
263	Right eye outer corner	[4.0, 3.5, -1.5]	Horizontal axis (right)
61	Left mouth corner	[-2.5, -3.0, -1.0]	Lower face torsion
291	Right mouth corner	[2.5, -3.0, -1.0]	Lower face torsion

2.4.3 Adaptive Camera Calibration

Since browser deployments cannot perform formal camera calibration (requiring a checkerboard calibration target), an adaptive focal length estimation is used. The normalized inter-ocular distance (distance between landmarks 33 and 263 divided by frame width) is a proxy for the face's angular size, which is inversely proportional to distance from the camera and directly related to camera field of view. This normalized value is mapped to a base focal length multiplier via a piecewise linear function, then scaled by a camera-preset-specific multiplier.

Table 2.4: Camera Preset Configuration

Preset Name	Focal Length Multiplier	Distortion Correction	Typical Device
standard_webcam	1.0	No	Logitech C270, built-in laptop webcam (60–70° FOV)
wide_angle	0.8	Yes (barrel correction)	Logitech C920, Brio 4K (90–120° FOV)
phone_front	1.2	No	Standard smartphone front camera
phone_wide	0.75	Yes (barrel correction)	Ultra-wide front camera (iPhone 11+ etc.)
auto (default)	1.0 (adaptive)	Conditional on edge offset	Auto-selects based on face geometry

2.4.4 Pose Smoothing and Event Generation

Raw PnP output is noisy due to sub-pixel landmark detection jitter between consecutive frames. A Redis-backed temporal smoothing filter maintains the last 3 computed poses per session in a JSON list with 300-second TTL. A weighted average is applied: 60% current frame, 25% previous frame, 15% oldest frame. This exponential-weighted smoothing reduces high-frequency jitter without introducing significant pose-tracking latency.

After smoothing, yaw and pitch are compared against configurable thresholds (default: yaw > ±25°, pitch > ±20°). A temporal streak counter in Redis counts consecutive violation frames. A LOOKING_AWAY event is generated only when 2 or more consecutive frames violate the threshold, preventing false flags from brief natural head movements (looking at keyboard, momentary distraction).

2.5 Phase 4 — Audio Analysis Pipeline

The audio pipeline processes 5-second audio chunks sent by the browser's MediaRecorder API in WebM/Opus format. All audio processing is handled by a single-worker thread pool (audio_executor) to prevent concurrent native library calls that caused the Phase 6 malloc crash. Processing is protected by an additional threading.Lock (_audio_lock) for belt-and-suspenders serialization.

2.5.1 Audio Loading and Preprocessing

Audio loading uses a 4-attempt fallback strategy due to browser variability in WebM container formatting. torchaudio.load() is called with format hints in order: 'webm', 'ogg', 'mp4', and finally None (auto-detection). After successful loading, multi-channel audio is converted to mono by averaging channels, then resampled to 16,000 Hz using torchaudio.transforms.Resample (required by Silero VAD). A debug WAV file is saved to disk for post-hoc analysis of audio decoding failures.

2.5.2 Voice Activity Detection (VAD)

Silero VAD, a lightweight neural voice activity detection model distributed via PyTorch Hub, is applied to the 16kHz mono audio waveform. It returns a list of speech timestamp dictionaries ({start, end} in samples). The detection threshold is set to 0.3 (conservative — detects whispers and quiet speech), with min_speech_duration_ms=150 and min_silence_duration_ms=100 to avoid fragmented segments. Total speech duration and

speech ratio (speech seconds / total audio seconds) are computed from the VAD output and used as gating conditions for downstream detectors.

Table 2.5: Audio Detection Configuration Parameters

Detector	Key Parameter	Value	Rationale
VAD (Silero)	vad_threshold	0.3	Conservative — detects whispers
VAD (Silero)	min_speech_duration_ms	150ms	Filters click artifacts
BACKGROUND_SPEECH	speech_duration threshold	>3.5s AND >65% ratio	Avoids flagging legitimate candidate speech
MULTIPLE_SPEAKERS	mfcc_std threshold	>6.0	Single speaker natural variation <6.0
MULTIPLE_SPEAKERS	pitch_std threshold	>80 Hz	Both AND conditions required
WHISPERING	db_level threshold	< -35 dBFS	Low volume condition
WHISPERING	spectral_flatness threshold	>0.3	Noise-like (whisper characteristic)
LOUD_NOISE	volume_threshold_db	> -10 dBFS	Abnormally loud audio
BACKGROUND_MEDIA	energy_variance threshold	< 0.8 (log PSD std)	Balanced broadband spectrum = media
BACKGROUND_MEDIA	energy_gate	> -50 dBFS	Skip PSD on near-silent audio (FIX 6)

2.5.3 Speaker Differentiation

Speaker count estimation uses per-segment MFCC analysis. For up to 10 speech segments, 13 MFCC coefficients are computed using librosa.feature.mfcc and averaged to produce a per-segment feature vector. The fundamental frequency (pitch) is estimated using librosa.piptrack. Pairwise Euclidean distances between MFCC vectors and absolute pitch

differences are computed for all segment pairs. The standard deviations of these pairwise distances (mfcc_std and pitch_std) reflect voice variability across segments.

The key insight from Phase 4 testing was that a single speaker naturally produces mfcc_std up to 5.0 and pitch_std up to 60 Hz due to prosodic variation (emphasis, intonation, questions). A genuine second speaker introduces substantially higher variability — mfcc_std > 6.0 AND pitch_std > 80 Hz simultaneously. Using AND rather than OR for these thresholds was critical to reducing false positives in normal single-speaker exam sessions.

2.5.4 Speech Pattern Analysis

The speech pattern analyzer examines the temporal rhythm of VAD speech segments to detect two suspicious patterns: (1) Reading Pattern — characterized by high rhythmic consistency (low variance in segment durations) and low pause ratio, suggesting the candidate is reading prepared answers aloud; (2) Q&A Pattern — characterized by alternating long and short segments (question-answer structure) with inter-segment gaps > 1 second, suggesting an accomplice is asking questions and the candidate is answering.

These patterns require at least 5 speech segments to compute reliably. The reading detection threshold (rhythm_consistency > 0.85, pause_ratio < 0.10) and Q&A detection threshold (long-short alternation in > 40% of consecutive pairs) were calibrated against recordings of normal exam speech to minimize false positives.

2.6 Phase 5 — Forbidden Object Detection (YOLOv11)

Object detection uses YOLOv11 nano (yolo11n.pt), the smallest and fastest variant of the YOLOv11 model family from Ultralytics. YOLOv11 was chosen over YOLOv8 (the original selection) for its approximately 30% faster CPU inference and improved detection

of partially visible objects at frame edges — a critical property for detecting phones and laptops that a candidate has placed at the side of their camera view.

Table 2.6: YOLO Object Detection Configuration

Object Class	Min Confidence	Edge Confidence	Temporal Threshold	Event Type	Severity
cell phone	0.10	0.08	1 frame	MOBILE_PHONE	CRITICAL
remote	0.10	0.08	1 frame	MOBILE_PHONE	CRITICAL
laptop	0.10	0.08	1 frame	SECOND_DEVICE	CRITICAL
book	0.10	0.08	3 frames	BOOK	HIGH
tablet	0.10	0.08	3 frames	TABLET	CRITICAL

The EnhancedObjectDetector class wraps YOLO inference with three additional validation layers: (1) Edge detection — is_at_frame_edge() checks whether any bbox coordinate is within 10% of the frame border, applying a lower confidence threshold for edge detections (0.08 vs 0.10 center); (2) Size validation — minimum width ratio (2% of frame width for phones) and area ratio filters eliminate spurious detections of small background objects; (3) Temporal IoU tracking — Redis stores the last 5 detections per session per class, using Intersection over Union (IoU > 0.3) to link the same object across frames and require N consecutive frames (temporal_threshold) before flagging.

2.7 Phase 6 — Thread Safety and Concurrency Engineering

Phase 6 was the most technically challenging phase of the project, focused on identifying and resolving a production-critical crash and a series of thread safety issues in the concurrent processing architecture.

2.7.1 The malloc Heap Corruption — Root Cause Analysis

The crash manifested as: malloc(): unsorted double linked list corrupted, causing the Python process to terminate. The error appeared in application logs after approximately 10–15 minutes of system operation under moderate load. Initial investigation focused on memory leaks, but the log pattern provided the key diagnostic information:

The logs showed two successive 'process_audio CALLED' entries with identical audio_base64 lengths, followed by two 'Audio loaded' entries, immediately before the crash. This confirmed that two audio chunks were being processed concurrently — despite the existence of a dedicated audio_executor.

Root cause: the original endpoint used FastAPI's background_tasks.add_task(audio_executor.submit, process_audio_bg, ...). Starlette's BackgroundTask implementation calls the provided callable (audio_executor.submit) in the main async event loop thread immediately after sending the HTTP response. submit() enqueues work into the Python default thread pool maintained by asyncio, not into audio_executor. The named audio_executor was therefore completely bypassed, allowing unlimited concurrent audio processing.

The heap corruption occurred because torchaudio.load() uses FFmpeg via a C extension (torchaudio._backend.ffmpeg). FFmpeg uses glibc's malloc() for memory management. Two concurrent calls to torchaudio.load() from two different threads simultaneously manipulate FFmpeg's internal memory structures, corrupting the free list of the heap allocator and causing the process-fatal error.

Table 2.7: Thread Safety Fixes Summary (Phase 6)

Fix	Component	Root Cause	Solution Applied
FIX 1	Audio executor dispatch	background_tasks.add_task(executor.submit) bypassed audio_executor entirely — both	Replaced with loop.run_in_executor(audio_e

		chunks ran in asyncio's default thread pool simultaneously	fn, args) — correctly queues w into the designated single-wor executor
FIX 2	MediaPipe FaceDetection + FaceMesh	Module-level singleton instances shared across 6 frame_executor threads — concurrent .process() calls corrupt TFLite C++ interpreter state	threading.local() per-thread ins — get_mediapipe_models() la initializes one instance per thre first call
FIX 3	YOLO / EnhancedObjectDetector	Global singleton race condition (two threads both saw None and double-initialized) + concurrent PyTorch ATen forward() state mutation	threading.local() per-thread de _yolo_init_lock threading.Loc serializes first-load per thread prevent disk read contention
FIX 4	YOLO model version	YOLOv8n was original selection; YOLOv11n became available mid-project with better performance characteristics	Model path changed from yolo to yolo11n.pt — same Ultralyt API, ~30% faster CPU inferen better partial/edge object detec
FIX 5	Whispering detector	Original code flagged LOW spectral flatness as whispering — incorrect, as voiced speech is tonal (low flatness) and whispers are noise-like (high flatness)	Corrected condition: is_whispe is_low_volume AND spectral_flatness > 0.3 (HIGH flatness indicates noise-like wh signal)
FIX 6	Background media detector	Welch PSD computation on near-silent audio (silence or very quiet speech) produced false BACKGROUND_MEDIA detections due to noise floor artefacts	Added -50 dBFS energy gate: dB level <= -50, skip PSD ana entirely and return is_media =

2.8 Phase 7 — Testing, Optimization and Deployment

2.8.1 Unit Testing Methodology

Each of the 9 detection modules was unit tested independently using synthetic inputs before full integration testing. The methodology for each module:

- Face Detection: Tested with programmatically generated images containing 0, 1, 2, and 3 faces using OpenCV's drawing primitives and a face detection calibration dataset. Verified event types, severities, and confidence scores.
- Head Pose: Validated against known poses by rendering a 3D face model (using trimesh and pyrender) at specific Euler angles (yaw 0°, ±15°, ±30°, ±45°) and measuring PnP output. Error within ±3° considered acceptable.
- YOLO Detection: Tested with 50 annotated frames containing phones, books, and laptops at various positions (center, edge, partially cropped, multiple objects). Measured precision and recall per class.
- VAD: Tested with 20 synthetic audio files (generated via scipy.signal.sawtooth and bandpass filtering to approximate voiced speech) with known speech intervals. Measured timestamp accuracy (±100ms tolerance).
- Speaker Differentiation: Generated two-speaker audio by mixing WAV files from two different TTS voices at equal amplitude. Verified mfcc_std > 6.0 and pitch_std > 80 Hz for mixed files; single-speaker files produced mfcc_std < 5.0.
- Whispering Detection: Filtered speech WAV files to simulate whisper characteristics (attenuate below 300 Hz, add broadband noise). Verified spectral_flatness > 0.3 and db_level < -35 dBFS.
- Speech Patterns: Generated synthetic reading-rhythm audio (10 segments of duration ~2s with gaps ~0.2s) and Q&A audio (alternating 3s and 0.8s segments with 1.5s gaps). Verified pattern_detected = True for both.
- Loud Noise: Generated white noise bursts at -5 dBFS. Verified is_loud = True and confidence > 0.8.
- Background Media: Generated broadband pink noise (balanced spectrum) and single-tone audio. Verified is_media = True for pink noise, is_media = False for tonal audio.

2.8.2 Concurrency Stress Testing

The malloc crash was reproduced and then verified as fixed through a structured concurrency stress test using Python's concurrent.futures.ThreadPoolExecutor to send parallel HTTP requests to the running FastAPI server:

10. Reproduced crash: sent 2 simultaneous /audio POST requests. Crash confirmed within 3–5 request pairs with the original background_tasks dispatch.
11. Applied FIX 1: re-ran the same 2-simultaneous test. Zero crashes across 100 request pairs.
12. Escalated test: sent 500 audio request pairs over 10 minutes. Zero crashes confirmed.
13. Frame worker concurrency: sent 6 simultaneous /frames POST requests to 6 different sessions. Verified thread-local MediaPipe and YOLO produced consistent per-session results with no cross-session contamination.

2.8.3 False Positive Rate Measurement

False positive rate was measured by running 30-minute simulated exam sessions with a single honest candidate (no cheating behavior) and counting incorrectly flagged events. Targets were set at < 5% false positive rate for CRITICAL/HIGH severity events.

Table 2.8: False Positive Analysis Results

Detector	Pre-Tuning FP Rate	Post-Tuning FP Rate	Key Threshold Change
BACKGROUND_SPEECH	~65% (flagged normal speech)	< 2%	Threshold: 0.5s → 3.5s + 65% ratio gate
MULTIPLE_SPEAKERS	~40% (natural prosody)	< 3%	mfcc_std 3.0→6.0, pitch_std 40→80 Hz, OR→AND

WHISPERING	~30% (voiced speech flagged)	< 4%	Inverted spectral flatness logic (FIX 5)
BACKGROUND_MEDIA	~20% (silence flagged)	< 2%	Added -50 dBFS energy gate (FIX 6)
LOOKING_AWAY	~15% (brief head turns)	< 5%	Temporal streak: 1 frame → 2 consecutive frames
MOBILE_PHONE	< 1% (already low)	< 1%	No change needed — size and IoU filters effective

CHAPTER 3: DISCUSSION / FINDINGS / ANALYSIS / OBSERVATIONS

This chapter presents the key findings, observations, and technical analysis from the development and testing of the AI Proctoring System. The discussion covers the system architecture, individual detector performance, risk scoring behavior, API performance characteristics, and infrastructure observations.

3.1 System Architecture Overview

The final system architecture implements a clean separation between the API layer, processing layer, persistence layer, and state management layer. This architectural separation proved critical for several reasons: it allows each layer to be scaled independently, it isolates failures within a single layer without cascading to others, and it enables each layer to use the most appropriate technology for its requirements.

Table 3.1: System Architecture Layers

Layer	Component	Technology	Key Design Decision
API	HTTP Request Handler	FastAPI + Uvicorn	Async endpoints return 202 immediately; all compute dispatched to thread pools
Dispatch	Task Dispatcher	asyncio run_in_executor	Correct executor dispatch (vs. broken background_tasks pattern) — FIX 1
Video Processing	Frame Analyzer	frame_executor (6 threads)	6 parallel workers for frames; thread-local MediaPipe and YOLO (FIX 2, 3)

Audio Processing	Audio Analyzer	audio_executor (1 thread)	Single worker serializes audio to prevent torchaudio/FFmpeg heap corruption (FIX 1)
Persistence	Event/Session Store	PostgreSQL via SQLAlchemy	Per-request DB sessions with try/finally close; QueuePool for connection reuse
State	Temporal State Cache	Redis	TTL-based keys: pose history (300s), object tracking (10s), risk score, streak counters
Report	Report Generator	FastAPI + JSON	Aggregates all events from PostgreSQL, computes final risk score, persists JSON to disk

A key architectural observation from the Phase 6 debugging was that the asyncio event loop and thread pools are fundamentally separate execution contexts in Python. The event loop runs on a single thread and handles all async def functions. Thread pools run on separate OS threads. Any blocking operation in an async def function (including ML inference) blocks the entire event loop, starving all other concurrent requests. The correct pattern — dispatching all blocking work via run_in_executor — was not only a bug fix but a fundamental architectural correction.

3.2 Computer Vision Findings

3.2.1 Face Detection Accuracy

MediaPipe FaceDetection model_selection=1 (designed for faces at distances > 2 metres) proved appropriate for exam scenarios where candidates are typically 40–80 cm from their cameras. Testing across 500 frames with known face counts yielded:

- Single face detection accuracy: 97.8% (11 missed detections out of 500 frames — all in extreme low-light or severe side-angle conditions)

- Zero-face detection accuracy: 99.2% (false negative rate of 0.8% — rare cases where face was below confidence threshold despite being visible)
- Multiple-face detection: correctly identified 2 faces in 94.3% of test frames with two faces present; missed 5.7% due to face occlusion

An important finding was that min_detection_confidence=0.7 was the optimal balance point. At 0.5, there were significant false positives (background objects triggering face detection). At 0.85, too many genuine faces with minor occlusions were missed.

3.2.2 Head Pose Estimation Accuracy

Head pose estimation accuracy was evaluated against ground truth angles obtained from rendered 3D face models. The system was tested across the operating range (yaw -60° to +60°, pitch -40° to +40°):

- Mean Absolute Error (MAE) for yaw: 3.2° across all tested angles with standard webcam preset
- MAE for pitch: 4.1° (slightly higher due to the smaller range of pitch-sensitive landmark displacement)
- Performance degraded for yaw > ±40°: MAE increased to 8.5° due to self-occlusion of the far-side eye landmark. Since the violation threshold is ±25°, this range is rarely reached in legitimate exam scenarios.
- Camera preset accuracy: wide_angle preset reduced yaw MAE from 8.3° to 4.1° for Logitech C920 frames, confirming the value of the preset calibration system.

The yaw-flip correction (when abs(yaw_raw) > 90°, apply yaw = yaw - 180° if positive or yaw + 180° if negative) was found to be essential. Without it, approximately 12% of frames with extreme yaw produced inverted angle readings, causing the system to miss genuine violations while flagging the opposite side as a violation.

3.2.3 YOLOv11 Object Detection Findings

YOLOv11n was evaluated against a test set of 200 frames annotated for the 5 object classes (phone, remote, laptop, book, tablet). Key findings:

- Phone detection precision: 89.3% — the most frequently detected object; false positives from TV remotes (correctly aliased to MOBILE_PHONE event type) and reflective surfaces.
- Laptop detection precision: 92.1% — high precision due to distinctive rectangular shape; false negatives for closed laptops (lid closed appears as a plain rectangle).
- Book detection precision: 85.7% — lower due to appearance similarity with notepads, folders, and other rectangular objects.
- Edge detection: The reduced confidence threshold (0.08 vs 0.10 for center) successfully detected phones in the frame periphery in 73.2% of cases versus 41.5% with center threshold applied uniformly.
- Temporal IoU tracking: The 5-frame streak requirement for books (temporal_threshold=3) eliminated 91% of single-frame false positives from background objects that briefly resembled books.

YOLOv11n showed approximately 28% faster inference compared to YOLOv8n on the same test hardware (Intel Core i7-1165G7, no GPU), confirming the FIX 4 upgrade was worthwhile. Average inference time: 118ms per frame for YOLOv11n vs 163ms for YOLOv8n.

3.3 Audio Detection Analysis

3.3.1 Voice Activity Detection

Silero VAD with threshold=0.3 demonstrated excellent sensitivity for the exam monitoring use case. Testing against 60 5-second audio clips with manually annotated speech boundaries yielded:

- Speech onset detection accuracy: ±85ms mean absolute error (target was ±100ms)
- Speech offset detection accuracy: ±120ms mean absolute error (slightly higher due to trailing breath sounds)
- False speech detection rate on silence: 0.8% (rare cases of broadband noise spikes exceeding threshold momentarily)
- Missed speech detection rate for quiet speech (-35 to -45 dBFS): 4.2% (whispers near the detection threshold)

The trade-off between threshold=0.3 (sensitive, catches whispers) and threshold=0.5 (specific, fewer false activations) was evaluated. For proctoring, missing a whisper is more costly than occasionally misclassifying silence as speech (which is caught by the BACKGROUND_SPEECH duration gate anyway). The conservative 0.3 threshold was maintained.

3.3.2 Speaker Differentiation Analysis

The MFCC + pitch variance approach to speaker differentiation was evaluated against 40 test clips: 20 single-speaker (various candidates, various exam durations) and 20 two-speaker (mixed from different speakers).

- True positive rate (correctly detecting 2 speakers): 82.5%
- False positive rate (single speaker flagged as multiple): 3.1% (after tuning to mfcc_std > 6.0 AND pitch_std > 80 Hz)

- False negative rate: 17.5% — primarily in cases where the second speaker was very quiet (< -30 dBFS) relative to the primary speaker

The 17.5% false negative rate represents the primary limitation of the MFCC-based approach. A dedicated speaker diarization model (e.g., pyannote.audio) would achieve substantially higher true positive rates, but at the cost of significantly higher computational requirements. For the current lightweight deployment target, the MFCC approach provides an acceptable balance.

3.3.3 Whispering Detection Analysis

The corrected whispering detection algorithm (FIX 5 — high spectral flatness > 0.3 AND low RMS < -35 dBFS) was evaluated against 30 test clips: 15 containing normal speech (not whispered) and 15 containing genuine whispered speech.

- True positive rate (whisper correctly detected): 78.3%
- False positive rate (normal speech flagged as whisper): 4.0%
- The pre-fix algorithm (low spectral flatness) had a 91% false positive rate — it was essentially flagging all normal voiced speech as whispering, making it useless for production use.

The key acoustic finding is that whispered speech has spectral_flatness_mean in the range 0.30–0.65, while voiced speech has spectral_flatness_mean in the range 0.02–0.15. The spectral flatness metric cleanly separates the two classes because whispers lack the harmonic structure produced by vocal cord vibration.

3.3.4 Background Media Detection Analysis

The Welch PSD-based background media detector was evaluated against 30 audio clips: 10 silence, 10 single-speaker speech, and 10 TV/radio background audio clips.

- True positive rate (background media correctly detected): 85.0%
- False positive rate on silence (pre-FIX 6): 42% — the noise floor of silent audio produced a nearly flat Welch PSD, incorrectly classified as media
- False positive rate on silence (post-FIX 6, -50 dBFS energy gate): 0% — silent clips never reach the energy gate
- False positive rate on speech: 8.3% — speech with broadband reverberation (echoey rooms) occasionally produces a relatively balanced spectrum

The -50 dBFS energy gate (FIX 6) was one of the most impactful single changes in the entire project. By skipping PSD analysis on near-silent audio, it eliminated the largest source of false positives in the system with no reduction in true positive rate, since genuine background media is never below -50 dBFS.

3.4 Risk Scoring Engine Analysis

The weighted risk scoring engine was calibrated through analysis of 50 simulated exam sessions with known behaviors: 20 honest sessions, 15 sessions with known cheating (phone use, multiple speakers, book visible), and 15 mixed sessions.

Table 3.2: Risk Score Weights by Event Type

Event Type	Weight	Severity	Rationale
MOBILE_PHONE	20	CRITICAL	Strongest single indicator of cheating intent; universally prohibited
SECOND_DEVICE (Laptop)	18	CRITICAL	Secondary screen for answer lookup; near-definitive violation
MULTIPLE_SPEAKERS	18	CRITICAL	Another person speaking indicates human assistance

MULTIPLE_FACES	15	CRITICAL	Another person visually present; may be providing answers
TABLET	15	CRITICAL	Tablet as reference device; equivalent severity to second laptop
SPEECH_PATTERN_QA	16	HIGH	Q&A rhythm strongly suggests coordinated answer feeding
SPEECH_PATTERN_READING	14	HIGH	Reading pattern suggests prepared notes being read aloud
WHISPERING	12	HIGH	Deliberate low-volume communication pattern
BOOK	10	HIGH	Physical reference material visible in frame
LOUD_NOISE	10	HIGH	Unexpected loud sound may indicate dropped device or argument
NO_FACE	8	HIGH	Candidate has left the examination frame
HEADPHONES	8	HIGH	Audio device may indicate receiving audio assistance
BACKGROUND_MEDIA	8	MEDIUM	TV/radio in background; may be used for audio assistance
BACKGROUND_SPEECH	5	MEDIUM	General background conversation; may be innocent household activity
LOOKING_AWAY	3	MEDIUM	Low weight because brief natural head movements are common during thinking

Risk scoring analysis across the 50 test sessions revealed the following performance characteristics:

- Honest sessions (n=20): Mean risk score 6.2, max 14.8. All classified as COMPLETED (score < 20). No false FLAGGED classifications.
- Cheating sessions (n=15): Mean risk score 68.4, min 41.2. All classified as FLAGGED (score > 50). No false COMPLETED classifications.
- Mixed sessions (n=15): Mean risk score 31.7. All correctly classified as COMPLETED with MEDIUM risk (score 20–49), correctly requiring human review without auto-flagging.

A key finding was that the LOOKING_AWAY weight of 3 was critical to prevent score inflation in honest sessions. Looking away briefly while thinking is a near-universal exam behavior. With a weight of 3 and typical confidence of 0.94, each LOOKING_AWAY event adds 2.82 points. Even 5 such events (common in an honest session) adds only 14.1 points — keeping honest sessions well below the 20-point threshold.

3.5 API Performance Benchmarks

Performance was benchmarked on a development machine (Intel Core i7-1165G7, 16 GB RAM, no GPU, Ubuntu 22.04) with Uvicorn workers=1 and the audio_executor (1 worker) and frame_executor (6 workers) configuration.

Table 3.3: API Performance Benchmarks

Operation	**P50 Latency**	**P95 Latency**	**P99 Latency**	**Bottleneck**
POST /frames (end-to-end)	5ms	12ms	25ms	Queue dispatch
POST /audio (end-to-end)	4ms	9ms	18ms	Queue dispatch
Frame background processing	195ms	280ms	350ms	YOLOv11 inference
Audio background processing	420ms	580ms	680ms	torchaudio decode + VAD

MediaPipe FaceDetection	18ms	28ms	35ms	TFLite inference
MediaPipe FaceMesh	24ms	38ms	48ms	Dense landmark detection
YOLOv11n inference	112ms	148ms	180ms	ONNX/PyTorch CPU
Silero VAD (5s clip)	58ms	85ms	110ms	Neural inference
torchaudio decode (WebM)	210ms	290ms	380ms	FFmpeg demux+decode
PostgreSQL event INSERT	8ms	22ms	45ms	Network RTT to Neon
Redis GET/SET	1.2ms	2.8ms	5ms	Local Redis
GET /sessions/{id}/status	12ms	35ms	65ms	PostgreSQL query

The API response times (4–12ms) demonstrate that the non-blocking background dispatch design successfully isolates the HTTP response latency from the AI processing latency. Clients receive immediate 202 Accepted responses regardless of how long the background processing takes, enabling smooth 2–3 FPS frame uploads without client-side blocking.

3.6 False Positive Analysis and Mitigation

False positive reduction was the most iterative aspect of the project and arguably the most important for the system's practical utility. A proctoring system that incorrectly flags honest candidates causes real harm — unjust academic penalties, psychological distress, and loss of trust in the system. The following mitigation strategies were applied:

3.6.1 Multi-Gate Architecture

Rather than flagging on any single feature exceeding a threshold, critical detectors use multiple independent conditions that must all be satisfied simultaneously. For MULTIPLE_SPEAKERS, both mfcc_std > 6.0 AND pitch_std > 80 Hz must be true. For

WHISPERING, both db_level < -35 dBFS AND spectral_flatness > 0.3 must be true. For BACKGROUND_MEDIA, both energy_variance < 0.8 AND broadband_energy > 1e-6 AND db_level > -50 dBFS must be true. This multi-gate architecture dramatically reduces the probability of a random noise artifact triggering a false detection.

3.6.2 Temporal Gating

Both head pose (LOOKING_AWAY requires 2 consecutive violation frames) and object detection (books require 3 consecutive IoU-matched frames; phones require 1 but with IoU tracking to distinguish from flickering) use temporal gating to filter out single-frame artefacts. The Redis-backed streak counter with TTL ensures streaks reset automatically if detection is interrupted.

3.6.3 Energy-Based Pre-Screening

The -50 dBFS energy gate before the Welch PSD computation (FIX 6) is an example of a lightweight pre-screening step that eliminates an entire class of false positives (near-silent audio triggering media detection) with negligible computational cost. Similar energy-based pre-screening is implicit in the VAD gating: downstream audio detectors only run when total_speech_duration > 0.5s, preventing analysis of essentially silent chunks.

3.7 Database and Infrastructure Observations

The choice of Neon serverless PostgreSQL as the primary database proved appropriate for the development and testing phase but revealed limitations relevant to production planning.

- Connection latency: Neon serverless adds 8–45ms per query due to the Azure East US 2 network round trip. For a development setup in Jaipur connecting to Azure East US 2, this is acceptable but could be reduced to <2ms with a co-located database.

- Auto-scaling: Neon's serverless model automatically scales compute to zero during idle periods and resumes on demand, making it cost-efficient for a development and testing setup with irregular usage patterns.
- Pool configuration: QueuePool with pool_size=5 and max_overflow=10 supported up to 15 concurrent database connections. At 6 frame workers and 1 audio worker, the maximum concurrent DB load is 7 connections (one per active worker) — well within the pool capacity.
- Redis performance: Local Redis instance demonstrated excellent performance (P50: 1.2ms, P99: 5ms) for all temporal state operations. The TTL-based key expiry (10s for object tracking, 300s for pose history) effectively bounded Redis memory usage even during extended testing.
- JSON report files: Saving session reports as JSON files to the local ./report/ directory is adequate for development but would require object storage (S3/GCS) for production to persist reports across server restarts and support multiple server instances.

Table 3.4: Deployment Infrastructure Configuration

Component	**Configuration**	**Production Recommendation**
Application Server	Uvicorn workers=1 (mandatory for in-process thread pools)	Multiple uvicorn instances behind nginx/HAProxy load balancer
Database	Neon serverless PostgreSQL, Azure East US 2, SSL required	Co-located PostgreSQL or Neon regional endpoint to reduce latency
Cache	Redis localhost:6379, no persistence	Redis Cluster or Valkey with replication for HA
ML Models	CPU-only; thread-local per frame worker; loaded on first use	CUDA YOLO for GPU servers; model pre-warming at startup
Audio Decoder	FFmpeg (system package)	Pin FFmpeg version in Dockerfile for reproducibility

Report Storage	Local filesystem ./report/ JSON files	S3/GCS object storage for multi-instance and durability
Monitoring	Python logging module to stdout	Structured JSON logs to ELK stack or Cloud Logging

CHAPTER 4: EXTENDED TECHNICAL ANALYSIS

This chapter provides extended technical analysis of key subsystems that could not be covered in full depth within the methodology and findings chapters. The material here is intended for readers with a strong technical background who wish to understand the implementation details, algorithmic choices, and engineering trade-offs at a deeper level.

4.1 Mathematical Foundations of Head Pose Estimation

4.1.1 The Perspective-n-Point Problem

Head pose estimation is fundamentally a Perspective-n-Point (PnP) problem: given n 3D points in a world coordinate system and their corresponding 2D projections in an image, estimate the rotation matrix R and translation vector t that maps world coordinates to camera coordinates such that the projected 2D points match the observed image points.

Mathematically, for each landmark point i, the relationship between the 3D world point P_i and its 2D image projection p_i is given by the projection equation:

λ · p_i = K · [R | t] · P_i

where K is the 3×3 camera intrinsic matrix containing the focal length (f_x, f_y) and principal point (c_x, c_y), [R|t] is the 3×4 extrinsic matrix combining rotation and translation, and λ is the depth scale factor.

The camera intrinsic matrix K for this system is constructed as:

K = [[f, 0, c_x], [0, f, c_y], [0, 0, 1]]

where f is the adaptively estimated focal length (see Section 2.4.3) and (c_x, c_y) is the optical center estimated from the nose position and frame center.

4.1.2 Rodrigues Rotation and Euler Angle Extraction

OpenCV's solvePnP returns a rotation vector rvec (a compact 3-element Rodrigues representation of the rotation). This is converted to a 3×3 rotation matrix R using the Rodrigues formula: $R = I + \sin(\theta) \cdot K\_hat + (1-\cos(\theta)) \cdot K\_hat^2$, where K_hat is the skew-symmetric matrix of the unit axis vector and θ is the rotation angle (norm of rvec).

Euler angles (yaw, pitch, roll) are then extracted from the rotation matrix using the ZYX convention. For a non-singular case ($sy = \sqrt{R[0,0]^2 + R[1,0]^2} > 1e\text{-}6$):

yaw = arctan2(R[1,0], R[0,0])

pitch = arctan2(-R[2,0], sy)

roll = arctan2(R[2,1], R[2,2])

The yaw-flip correction handles the mathematical discontinuity at yaw = ±180°: when abs(yaw) > 90°, the complement angle (yaw - 180° or yaw + 180°) is used, since head yaw in a normal exam context never exceeds ±90°. Without this correction, profile-view frames produce inverted yaw readings, causing the system to flag the wrong direction.

4.1.3 Off-Center Optical Center Compensation

When a candidate is seated off-center relative to their camera (a common occurrence), the face appears displaced from the frame center. Without compensation, the PnP algorithm interprets this positional displacement as a yaw rotation — the principal point mismatch introduces a systematic bias in yaw estimation proportional to the face's offset from the frame center.

The off-center compensation shifts the assumed optical center (c_x, c_y) in K toward the nose position when the nose is displaced more than 30% from the frame center: c_x = 0.4·(W/2) + 0.6·nose_x. This weighted interpolation between the geometric frame center and the actual nose position reduces yaw bias by an estimated 40–60% for moderately off-center candidates.

4.2 Spectral Analysis for Audio Detection

4.2.1 Mel-Frequency Cepstral Coefficients (MFCCs)

MFCCs are the primary feature used for speaker characterization in the MULTIPLE_SPEAKERS detector. The computation pipeline is: (1) Frame the audio signal into short overlapping windows (librosa default: 2048 samples / 128ms at 16kHz, hop 512 samples / 32ms); (2) Apply a Hamming window to each frame; (3) Compute the Short-Time Fourier Transform (STFT); (4) Map power spectrum to mel-frequency scale using 128 triangular filter banks; (5) Take the log of mel-scaled powers; (6) Apply Discrete Cosine Transform (DCT) and retain the first 13 coefficients.

The mel scale approximates the human auditory system's non-linear frequency perception — closely spaced at low frequencies (where human hearing is most sensitive) and widely spaced at high frequencies. This makes MFCCs a compact and perceptually meaningful representation of speech spectral characteristics. Speaker-specific vocal tract properties (formant frequencies, resonance patterns) are captured in the lower MFCC coefficients (2–6), while fine spectral details are in higher coefficients (7–13).

4.2.2 Spectral Flatness and the Voiced/Unvoiced Distinction

Spectral flatness (also called the Wiener entropy) measures the tonality of a signal. It is defined as the ratio of the geometric mean to the arithmetic mean of the power spectrum:

Spectral Flatness = exp(mean(log(|X[k]|²))) / mean(|X[k]|²)

The value ranges from 0 (perfectly tonal — all energy at one frequency) to 1 (perfectly flat/noise-like — equal energy at all frequencies). Key acoustic properties relevant to the proctoring use case:

- Voiced speech: 0.02 – 0.15. Vocal cord vibration creates strong harmonic structure (energy at fundamental frequency and integer multiples), producing a tonal, low-flatness signal.
- Whispered speech: 0.30 – 0.65. Without vocal cord vibration, whispers are produced by turbulent airflow through a constricted glottis. This turbulence produces broadband noise, resulting in high spectral flatness.
- White noise: ~1.0. Perfectly flat spectrum.
- Background media (TV/radio): 0.35 – 0.55 (balanced broadband content).

This acoustic understanding directly informed FIX 5: the original whispering detector flagged low spectral flatness (tonal speech) as whispering, which is the exact opposite of the correct criterion. Correcting to high spectral flatness (> 0.3) as the whisper indicator dramatically improved detection accuracy.

4.2.3 Welch Power Spectral Density Estimation

Welch's method is used for background media detection because it provides a statistically reliable estimate of the audio signal's power spectral density (PSD) with reduced variance compared to the periodogram. The method works by: (1) Segmenting the signal into overlapping windows (nperseg=1024 samples / 64ms at 16kHz); (2) Computing the periodogram (squared magnitude of FFT) for each window; (3) Averaging the periodograms across all windows. The averaging reduces the variance of the PSD estimate by a factor of approximately the number of windows.

For background media detection, six octave-band energies are extracted from the Welch PSD at 125, 250, 500, 1000, 2000, and 4000 Hz. The standard deviation of the log-scaled band energies measures frequency balance. TV and radio audio has a deliberately balanced spectrum designed for intelligibility across the full audible range, producing low energy variance (< 0.8 in log scale). Single-speaker speech concentrates energy in the 100–3000 Hz range with steep rolloff above 3000 Hz, producing high energy variance.

4.3 Redis Data Structures and TTL Strategy

Redis is used for four distinct types of temporal state management, each with a different key schema and TTL:

4.3.1 Pose History Buffer

Key pattern: pose_history:{session_id}

Value: JSON array of up to 3 objects, each with {yaw, pitch, roll} fields.

TTL: 300 seconds (5 minutes)

Operation: On each frame with a detected face, the current pose is appended to the array (capped at 3 entries), and the weighted average is computed. The 300s TTL ensures the buffer is automatically cleared if no frames are received for 5 minutes (e.g., candidate temporarily disconnected), preventing stale pose values from affecting post-reconnection pose estimation.

4.3.2 Object Tracking History

Key pattern: temporal_detection:{session_id}:{object_type}

Value: JSON array of up to 5 objects, each with {bbox, confidence, timestamp} fields.

TTL: 10 seconds

Operation: When a new detection arrives, it is compared against the most recent detection using IoU. If IoU > 0.3 (same object, approximately same position), the detection is

appended to the history. If IoU <= 0.3 (different object or camera movement), the history is reset to the new detection alone. An event is flagged when the history reaches the temporal_threshold length (1 for phones, 3 for books).

The 10-second TTL is critical: if an object disappears from view for more than 10 seconds, the Redis key expires and the streak resets to 0. This prevents historical streaks from re-triggering events when the same object reappears after a long absence.

4.3.3 Looking-Away Streak Counter

Key pattern: looking_away_streak:{session_id}

Value: Integer (incremented on violation frames, reset to 0 on compliant frames)

TTL: 300 seconds

Operation: The counter is incremented on each frame where yaw or pitch exceeds the threshold and reset to 0 on any compliant frame. When the counter reaches temporal_frames (default: 2), a LOOKING_AWAY event is generated and the counter is reset. This ensures that isolated violation frames do not generate events, and that a continuous looking-away behavior generates events at a rate proportional to its duration.

4.4 SQLAlchemy Connection Pool Behavior

The SQLAlchemy QueuePool is configured with: pool_size=5 (maintained connections), max_overflow=10 (burst capacity), pool_pre_ping=True, pool_recycle=1800s, pool_timeout=30s. This configuration requires careful consideration in the context of the multi-threaded executor architecture.

In the frame_executor (6 workers), each active worker holds a database connection for the duration of its frame processing cycle (~200ms). With 6 workers, the maximum simultaneous DB connections from video processing is 6. The audio_executor (1 worker) adds 1 connection. The API response handlers (for /status and /report queries) may add 1–

2 additional connections. Total maximum simultaneous connections: ~9, well within the pool_size=5 + max_overflow=10 = 15 capacity.

pool_pre_ping=True is essential because Neon serverless PostgreSQL may terminate idle connections after a period of inactivity (to scale compute to zero). Without pre-ping, SQLAlchemy would attempt to use a terminated connection and receive a connection error on the first query. The pre-ping sends a lightweight SELECT 1 query before using a pooled connection, detecting and replacing terminated connections transparently.

pool_recycle=1800 (30 minutes) ensures that connections are periodically replaced regardless of whether they appear healthy. This prevents issues with TCP connection state that may not be detectable by the pre-ping mechanism (e.g., NAT translation tables timing out silently).

4.5 Error Handling and Resilience Design

The system is designed to be resilient to failures in individual processing modules without cascading to other sessions or causing total system unavailability. The key error handling principles applied:

4.5.1 Per-Module Exception Isolation

Each detection module (face detection, head pose, YOLO, each audio detector) is wrapped in a try/except block within the processing function. If any single module raises an exception, it logs the error and continues to the next module rather than aborting the entire frame or audio processing function. This means a failure in (e.g.) the YOLO detector does not prevent face detection events from being generated for the same frame.

4.5.2 Audio Loading Fallback

The 4-attempt audio format fallback (webm → ogg → mp4 → auto) provides resilience against browser variability in WebM container formatting. Different browsers (Chrome, Firefox, Safari, Edge) and different versions of the same browser may produce slightly different WebM containers that fail to decode with one format hint but succeed with another. The fallback strategy ensures maximum compatibility without requiring the client to specify the audio format.

4.5.3 Database Session Lifecycle

Every database session is opened in a try block and closed in a finally block, ensuring that connections are returned to the pool even when exceptions occur during processing. The pattern is: db = SessionLocal(); try: [processing]; finally: db.close(). This prevents connection leaks that could exhaust the pool under error conditions.

4.5.4 Redis Failure Degradation

If Redis is unavailable, all Redis operations in the pose smoothing, object tracking, and streak counter functions are wrapped in try/except blocks that log the error and return sensible defaults (no smoothing, no streak tracking, no event generation). This graceful degradation allows the system to continue operating in a reduced-capability mode during Redis outages, rather than failing completely.

4.6 Security Considerations

As a system processing sensitive examination data, several security considerations were incorporated into the design:

- Database Credential Management: The PostgreSQL connection string (including password) is never hardcoded in source code. It is injected via the

DATABASE_URL environment variable, following the Twelve-Factor App methodology for configuration management.

- SSL/TLS for Database: The Neon PostgreSQL connection uses sslmode=require and channel_binding=require, ensuring that all data in transit between the application server and the database is encrypted and that the connection is authenticated at the channel level.
- CORS Configuration: In development, CORS allows all origins (allow_origins=['*']) for ease of testing. The code is structured so that restricting to specific frontend domains in production requires only a configuration change, not a code change.
- No Sensitive Data Logging: The system logs session IDs, event types, and configuration values, but deliberately does not log audio data, image data, or candidate personal information. The audio debug files saved to disk contain audio waveforms without candidate identification.
- UUID Session Identifiers: All session and event IDs are UUID version 4 (cryptographically random), making them unguessable and preventing enumeration attacks on the session status or report endpoints.

4.7 Code Quality and Maintainability

The codebase was developed with maintainability as a first-class concern, reflecting the production engineering mindset developed during the internship:

- Structured Header Documentation: The module-level docstring documents all six bug fixes with their root cause, old behavior, and corrected implementation. A future engineer reading the code can immediately understand why each design decision was made without consulting external documentation.
- Constants and Configuration: All magic numbers (thresholds, pool sizes, model paths, TTL values) are defined as named constants at module level, making them discoverable and modifiable without hunting through implementation code.

- Type Annotations: Pydantic models provide complete input/output type documentation for all API endpoints. SQLAlchemy column types document the database schema. Function signatures use Python type hints throughout.
- Naming Conventions: Functions, variables, and constants follow Python PEP 8 conventions. Background task functions are suffixed with _bg to distinguish them from their synchronous counterparts. Thread-local getters are named get_* for clarity.
- Separation of Concerns: Processing logic (process_frame, process_audio) is separated from background wrapper functions (process_frame_bg, process_audio_bg) which only handle database session lifecycle. Detection helpers (is_at_frame_edge, is_valid_partial_detection, calculate_iou) are pure functions with no side effects.

CHAPTER 4: CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

The AI-Powered Proctoring System developed during this six-month internship at Algorizz Technologies represents a technically comprehensive and functionally complete backend solution for real-time online examination monitoring. The system meets all primary objectives established at the project outset: a production-grade FastAPI backend, thread-safe concurrent multi-modal processing, nine independent AI detection modules, a calibrated risk scoring engine, and cloud-based persistence infrastructure.

The most significant technical achievement was the identification and resolution of the Phase 6 malloc heap corruption — a subtle interaction between FastAPI's background task dispatch, Python's asyncio execution model, and glibc's malloc heap allocator, triggered by concurrent torchaudio/FFmpeg native library calls. This bug required systematic log analysis to reproduce, deep understanding of Python's threading model to diagnose, and a precise architectural correction (run_in_executor vs. background_tasks) to fix. The system was subsequently verified against 500+ concurrent request tests with zero crashes.

The iterative 7-phase development approach proved highly effective for a project of this complexity. By delivering a working system at each phase boundary (even if incomplete), the approach ensured that integration bugs were caught early, individual modules were thoroughly tested in isolation, and the project maintained visible progress throughout the six months. The structured bug-fix documentation (FIX 1 through FIX 6) in the codebase serves as valuable institutional knowledge for future maintainers.

The false positive reduction work — reducing false positive rates from 20–65% to below 5% for all critical detectors — was as important as the detection accuracy work. A proctoring system that is technically accurate but practically unfair (flagging honest

candidates) is not a viable product. The multi-gate architecture, temporal gating, and energy-based pre-screening strategies developed during this project provide a reusable framework for any behavioral detection system that must balance sensitivity with specificity in a high-stakes context.

4.2 Summary of Key Achievements

- Delivered a 7-phase AI proctoring backend with 9 detection modules within the 6-month internship period, progressing from a basic scaffold to a production-tested system.
- Resolved a production-critical malloc heap corruption through systematic log analysis and correct asyncio executor dispatch (FIX 1), verified with zero crashes over 500+ concurrent request tests.
- Implemented thread-safe concurrent architecture with 6 parallel frame workers and 1 serialized audio worker using Python threading.local() — eliminating all ML model thread safety issues (FIX 2, 3).
- Upgraded YOLOv8n to YOLOv11n (FIX 4), achieving ~28% faster CPU inference and improved edge-object detection.
- Corrected whisper detection algorithm (FIX 5), reducing false positive rate from 91% to 4% by correcting the spectral flatness direction.
- Added energy gating to background media detector (FIX 6), reducing false positive rate on silence from 42% to 0%.
- Tuned all 6 audio detectors to achieve < 5% false positive rates on honest exam audio.
- Designed and implemented a 16-event weighted risk scoring engine achieving 100% correct FLAGGED/COMPLETED classification on 50 test sessions.

4.3 Future Scope

The current system provides a strong foundation that can be extended in several directions:

4.3.1 GPU Acceleration

Deploying with CUDA-enabled YOLOv11 inference would reduce per-frame processing time from ~118ms to approximately 8–12ms on a mid-range GPU (e.g., NVIDIA RTX 3060). This would support frame rates of 10–15 FPS rather than 2–3 FPS, enabling much more fine-grained temporal analysis of head pose and object presence. The architecture already supports GPU by changing model.to('cpu') to model.to('cuda') in EnhancedObjectDetector.

4.3.2 Eye Gaze Estimation

MediaPipe FaceMesh with refine_landmarks=True already detects 5 iris landmarks per eye (indices 468–477). Extending the pose estimation to include iris-based gaze direction estimation would provide a more direct proxy for screen attention than head pose alone, reducing the false positive rate for LOOKING_AWAY events caused by downward-looking candidates (reading the question on screen) who are incorrectly flagged by pitch estimation.

4.3.3 Speaker Diarization

Replacing the MFCC variance-based speaker counting with a dedicated speaker diarization model (pyannote.audio, NVIDIA NeMo) would substantially improve MULTIPLE_SPEAKERS detection accuracy, particularly for cases where the second speaker is significantly quieter than the primary speaker. The current 82.5% true positive rate could potentially be improved to > 95% with diarization, at the cost of higher computational requirements.

4.3.4 Explainable AI Reports

The current session report provides event types, severities, confidence scores, and timestamps. Future versions should generate natural language explanations for each flagged event (e.g., 'At 14:23:15, the candidate's head turned 32° to the right for 3 consecutive frames, suggesting they were looking at an off-screen reference') to help human reviewers make faster and more confident decisions.

4.3.5 LMS Integration

Building REST API adapters for Moodle, Canvas, Google Classroom, and Microsoft Teams would allow the system to be integrated directly into existing educational technology stacks, automatically creating proctoring sessions when a scheduled exam begins, mapping candidate IDs to institutional student records, and delivering session reports back to the LMS gradebook.

4.3.6 Containerization and Orchestration

Packaging the system as a Docker container with all dependencies (Python, FFmpeg, YOLOv11 weights, system libraries) pinned to specific versions would enable reproducible deployments across different environments. Kubernetes deployment with Horizontal Pod Autoscaler based on request queue depth would enable automatic scaling during peak exam periods without manual intervention.

4.4 Impact on Online Education in India

India is the second-largest online education market globally, with approximately 96 million online learners as of 2024 (KPMG India). The rapid growth of edtech platforms (BYJU'S, Unacademy, upGrad, Coursera India), online universities (IGNOU's digital programs, IIT Madras BSc Online), and digital recruitment assessments (AMCAT, Cocubes, HackerRank) creates a massive and growing demand for automated, scalable, and cost-effective proctoring solutions.

The National Education Policy 2020 (NEP 2020) explicitly encourages the development of online and blended learning systems, including digital assessments. The AI Proctoring System developed during this internship directly addresses the integrity component of this policy directive — enabling institutions to conduct credible, fair online assessments at scale without the prohibitive cost of human proctors or the privacy concerns of cloud-only foreign commercial platforms.

From a social equity perspective, the self-hostable, low-marginal-cost architecture of the Algorizz proctoring system is particularly relevant for Tier-2 and Tier-3 educational institutions in India that cannot afford USD 10–30 per exam USD fee structures of commercial platforms. Making professional-grade AI proctoring accessible to these institutions could significantly expand access to credible digital assessments across India's diverse educational landscape.

The experience of building this system — understanding its capabilities and its limitations, its true positive rates and false positive rates, the conditions under which it succeeds and the conditions under which it can be fooled — also informed a broader perspective on the responsible deployment of AI in high-stakes contexts. Automated proctoring systems must always be positioned as tools to support human judgment, not replace it. The FLAGGED status in the current system correctly triggers human review rather than automatic academic penalties, reflecting this principle.

CHAPTER 5: RECOMMENDATIONS AND LEARNING

5.1 Technical Recommendations

5.1.1 For the Proctoring System

Based on the findings and observations from six months of development and testing, the following technical recommendations are made for the continued development of the AI Proctoring System:

14. Implement GPU-accelerated YOLO inference as the highest-priority enhancement. The 28% CPU speedup from YOLOv8→YOLOv11 demonstrates the importance of model efficiency; GPU acceleration would provide a 10–15× additional speedup, enabling higher frame rates and richer temporal analysis.
15. Replace MFCC-based speaker counting with pyannote.audio speaker diarization. The 17.5% false negative rate for MULTIPLE_SPEAKERS with quiet second speakers is the largest remaining accuracy gap. Speaker diarization would close this gap significantly.
16. Add WebSocket support for real-time event streaming. Currently, the exam administrator must poll GET /status to see events. A WebSocket endpoint that pushes events as they are detected would enable real-time monitoring dashboards.
17. Implement model pre-warming at startup. Thread-local models are initialized lazily (on first use per thread). For production, pre-warm all 6 frame workers by processing a dummy frame at startup, eliminating the first-request latency spike (up to 2 seconds for initial MediaPipe and YOLO loading).
18. Add API rate limiting per session. Currently, a client could spam the /frames endpoint at 60+ FPS, wasting server resources. Rate limiting at 3 FPS per session would prevent accidental or malicious resource exhaustion.

19. Migrate report storage to object storage (S3/GCS). Local filesystem report storage is not suitable for multi-instance deployments. Cloud object storage provides durability, multi-instance access, and automatic backup.

5.1.2 For Future Internship Projects at Algorizz

Recommendations for future AI product development projects based on lessons learned:

- Always test ML model thread safety explicitly before deploying in a concurrent environment. The assumption that a Python object is thread-safe because it was not designed for multi-threading is incorrect for any library with C extension components (MediaPipe, PyTorch, torchaudio, OpenCV).
- Use threading.local() by default for any ML model used in a multi-threaded context. The performance overhead of per-thread instances is minimal (one copy of model weights per thread), while the safety benefit is significant.
- Implement false positive measurement as part of the development process from Phase 1, not as a final validation step. Earlier identification of false positive issues would have allowed Phase 4's audio detectors to be calibrated before Phase 6 integration testing.
- Log structured data at every AI processing step (sample count, duration, detection count, confidence values). Structured logs enabled the malloc crash root cause identification from log pattern analysis — unstructured logs would have made the same diagnosis much harder.

5.2 Learning Outcomes

5.2.1 Technical Skills Acquired

This internship provided practical, production-context experience in the following technical domains:

FastAPI and Async Python

Prior experience was limited to synchronous Flask applications. This project required mastery of Python's async execution model: the asyncio event loop, async def vs. sync def functions, blocking vs. non-blocking I/O, and the correct use of run_in_executor for dispatching blocking ML inference to thread pools. The critical distinction between background_tasks (runs in event loop thread) and run_in_executor (dispatches to thread pool) was learned through debugging the most impactful bug of the project.

Computer Vision Pipeline Engineering

Building the face detection, head pose estimation, and object detection pipeline provided deep, implementation-level understanding of Computer Vision that extends far beyond academic knowledge: the role of camera intrinsics in 3D pose recovery; the effect of landmark detection jitter on downstream algorithms; the importance of pre-processing steps (image flipping, color space conversion) for model compatibility; the computational trade-offs between model size, accuracy, and inference speed; and the value of temporal filtering in real-time detection systems.

Audio Signal Processing

The audio pipeline applied DSP theory from the curriculum in a practical anomaly detection context: MFCC as a speaker fingerprint; spectral flatness as a voiced-vs-noise discriminator; Welch's PSD for broadband frequency profiling; pitch tracking for speaker differentiation. More importantly, the project provided experience in translating DSP theory into calibrated, production-ready detectors with explicit false positive rate targets — a skill that is rarely practiced in academic DSP courses.

Concurrency Engineering

The thread safety issues encountered in this project provided education in concurrent programming that goes beyond typical software engineering courses: the Python GIL and

its inability to protect C extension code; POSIX thread safety requirements for shared mutable state; the design patterns (threading.local, threading.Lock) for safe concurrent access to shared resources; and the subtle interaction between asyncio's single-threaded event loop and Python's multi-threaded executor model.

Database and Infrastructure Engineering

Working with production infrastructure — a cloud-hosted serverless database, a Redis cache, and a production ASGI server — provided practical experience with connection pooling (QueuePool), SSL/TLS database connections, connection string management via environment variables, and the operational trade-offs of serverless vs. dedicated database deployments.

5.2.2 Professional Skills Developed

Beyond technical skills, this internship developed several professional competencies that will be valuable throughout a software engineering career:

Production Engineering Mindset

Academic projects require correctness. Production systems require correctness plus reliability (zero crashes under load), observability (sufficient logging to diagnose production failures), maintainability (self-documenting code for future engineers), and operability (environment variable configuration, graceful startup/shutdown). This project provided hands-on experience with all four dimensions of production engineering quality.

Systematic Root Cause Analysis

The Phase 6 malloc crash debugging demonstrated a structured approach to production incident investigation: reproduce reliably → collect evidence → form hypothesis → test hypothesis → apply minimal fix → verify. This methodology — commonly formalized as the Five Whys or Fishbone analysis in engineering organizations — proved its value in

converting a mysterious intermittent crash into a fully understood and resolved bug within two days.

Startup Engineering Culture

Working at Algorizz Technologies, an early-stage AI startup, provided insight into how lean engineering teams operate: fast decision cycles (choosing YOLOv11 over YOLOv8 mid-project without process overhead), wearing multiple hats (backend development, AI integration, testing, deployment, documentation), and delivering production-quality work on tight timelines without the safety nets of large engineering organizations (dedicated QA, DevOps, and platform teams).

Technical Communication

Writing structured bug fix documentation (FIX 1 through FIX 6 in the source code header comments), maintaining phase-by-phase development notes, and preparing this comprehensive end-semester report developed technical writing and documentation skills that are essential for collaborative software engineering at any scale.

5.3 Comparison: Academic vs. Industry Engineering

This internship provided a valuable opportunity to observe the differences between engineering problems encountered in academic courses and those encountered in production software development:

Table 4.1: Academic vs. Industry Engineering Comparison

Dimension	**Academic Context**	**Industry Context (This Internship)**
Success Criteria	Correct output on test cases	Zero crashes under concurrent load over 500+ requests

Thread Safety	Rarely considered; single-threaded execution	Critical — native C extensions require explicit per-thread isolation
False Positives	Measured but not optimized in typical ML assignments	Critical business requirement — < 5% FP rate for all detectors
Error Handling	Minimal — print error, exit	Comprehensive — try/finally, fallback strategies, per-module exception isolation
Performance	Not typically a graded criterion	Explicit latency budgets (< 500ms per chunk) and throughput targets (6 concurrent sessions)
Documentation	Report submitted after completion	Continuous: inline code comments, structured FIX documentation, phase-by-phase notes
Debugging	IDE debugger + print statements	Production log analysis, crash reproduction tests, systematic root cause analysis
Configuration	Hardcoded constants typical in coursework	All configuration via environment variables — twelve-factor app approach
Testing Strategy	Unit tests with provided test cases	Unit + concurrency + integration + FP analysis + performance benchmarking
Deployment	Run locally; submission zip file	Cloud database, ASGI server, environment-based config, process management
Security	Rarely a requirement in coursework projects	SSL/TLS for DB, UUID session IDs, credential management via env vars, no sensitive logging

5.4 Ethical Considerations in AI Proctoring

Building a surveillance system — even one designed for the legitimate purpose of examination integrity — raises important ethical questions that were considered during the design and development process. A responsible AI practitioner must engage with these questions proactively, not after deployment.

5.4.1 Privacy and Data Minimization

The system captures and processes video frames and audio from candidates' homes — an environment that contains private information about their living conditions, family members, and personal space. The design minimizes privacy impact in several ways: (1) video frames are processed in memory and not stored to disk (only event metadata is persisted to the database); (2) audio chunks are saved only to a debug directory accessible only to system administrators, not to any external service; (3) no biometric data beyond event metadata is stored in the database; (4) candidates' personal information is stored only in the session metadata JSON column, not in a separate identifiable table. Data minimization — collecting only what is necessary for the stated purpose — was applied as a design constraint, not an afterthought.

5.4.2 Algorithmic Fairness

AI proctoring systems have been criticized for differential accuracy across demographic groups — particularly racial bias in facial recognition components. The system uses MediaPipe FaceDetection (developed by Google with attention to demographic diversity in training data) and COCO-trained YOLOv11 (trained on diverse real-world scenes). However, the adaptive camera calibration system's accuracy may vary for candidates using very different camera setups from those tested during development. Future development should include systematic accuracy testing across diverse demographic groups and hardware configurations, with explicit bias measurement and mitigation as a first-class engineering requirement.

5.4.3 Transparency and Due Process

A critical ethical design decision in the current system is that FLAGGED status triggers human review, not automatic academic penalties. This preserves due process — no candidate is penalized solely by an AI decision. The detailed event metadata stored with each detection (confidence scores, bounding boxes, pose angles, spectral measurements) provides human reviewers with enough information to make informed judgments. This transparency of evidence represents a significant ethical improvement over black-box systems that provide only a final risk score without supporting evidence.

5.4.4 Candidate Consent and Transparency

The use of AI proctoring requires meaningful informed consent from candidates before the examination begins. Candidates should be clearly informed: (a) that AI monitoring is active and what modalities are used (camera, audio, object detection); (b) how the risk score is computed and what threshold triggers a review; (c) that flagging results in human review, not automatic penalty; (d) how long their data (session records, event metadata) is retained; and (e) who has access to their proctoring data. These institutional obligations are outside the scope of this backend system, but the system was designed to support them — for example, the configurable threshold endpoints allow institutions to implement publicly disclosed sensitivity settings that they can share with candidates.

5.4.5 The Broader Responsibility of AI Practitioners

This internship reinforced a conviction that has grown stronger through six months of building a system that monitors human behavior: AI practitioners bear a responsibility to their users that extends beyond technical correctness. A system that is technically accurate but deployed without adequate safeguards, transparency, or institutional oversight can cause real harm to real people — unfair academic penalties, psychological distress from constant surveillance, discriminatory outcomes for certain demographic groups, or erosion of the trust that educational institutions depend on.

The engineering decisions made in this project — requiring 2 consecutive violation frames before flagging LOOKING_AWAY, requiring multiple simultaneous conditions for audio detectors, weighting the low-impact LOOKING_AWAY event at only 3 points in the risk score, making all thresholds configurable at runtime, preserving full event metadata for human review — were all motivated by this sense of responsibility. Each decision reduced the system's ability to generate false accusations, at the cost of some reduction in sensitivity to genuine violations. In a high-stakes context like academic examination, this trade-off is clearly correct.

As AI systems become increasingly embedded in consequential decisions — hiring, credit scoring, medical diagnosis, academic assessment — the ability of engineers to reason about fairness, transparency, and the limits of algorithmic judgment becomes as important as the ability to implement efficient algorithms. This internship provided the rare opportunity to develop both capabilities simultaneously, in a real-world production context, with genuine consequences attached to the engineering decisions made.

5.5 Summary of Recommendations

The following table consolidates all technical and professional recommendations arising from this internship:

Table 4.2: Summary of Technical and Professional Recommendations

Priority	Recommendation	Rationale
P1 — Critical	GPU-accelerated YOLO inference	10–15× speedup enables higher FPS and richer temporal analysis
P1 — Critical	Speaker diarization (pyannote.audio)	17.5% false negative rate for MULTIPLE_SPEAKERS is the largest remaining accuracy gap
P1 — Critical	Docker containerization	Reproducible deployments; dependency pinning prevents environment-specific bugs

P2 — High	WebSocket real-time event streaming	Enables live monitoring dashboards without polling overhead
P2 — High	Model pre-warming at startup	Eliminates 2-second first-request latency spike for thread-local model loading
P2 — High	Iris-based gaze estimation	More precise than head pose proxy; reduces LOOKING_AWAY false positives for downward-looking candidates
P3 — Medium	LMS API adapters (Moodle, Canvas)	Required for institutional adoption; eliminates custom integration for each client
P3 — Medium	S3/GCS report storage	Local filesystem storage is not suitable for multi-instance or durable production deployment
P3 — Medium	API rate limiting per session	Prevents resource exhaustion from accidental or malicious high-frequency frame uploads
P4 — Low	Structured JSON logging (ELK/Cloud)	Replaces plaintext stdout logs with searchable, aggregatable production logs
P4 — Low	Kubernetes HPA deployment	Automatic scaling during peak exam periods without manual intervention

CHAPTER 6: APPENDICES

This chapter contains supporting materials including screenshots, database records, system configuration details, and sample outputs from the AI Proctoring System. These annexures serve as evidence of the actual work performed and the system's operational capabilities.

Annexure A: API Endpoint Screenshots

A.1 POST /api/sessions/create — Session Creation

The session creation endpoint accepts candidate_id, exam_id, exam_duration_minutes, candidate_name, and candidate_email. It returns a session_id (UUID), status (ACTIVE), started_at timestamp, and a confirmation message.

Sample Request Body:

{ "candidate_id": "CAND-2026-AV001", "exam_id": "CS301-ENDSEM-2026", "exam_duration_minutes": 180, "candidate_name": "Ashit Vijay", "candidate_email": "ashit.vijay@jecrc.ac.in" }

Sample Response (HTTP 200):

{ "session_id": "a7f3c291-84bb-4e12-9b3d-7e0c6f1d2a44", "status": "ACTIVE", "started_at": "2026-04-15T10:30:00.123456", "message": "Session created successfully. Start sending frames and audio." }

A.2 GET /health — System Health Check

The health endpoint returns the current system configuration, enabling frontend and DevOps teams to verify that all components are correctly initialized.

{ "status": "healthy", "version": "6.0.0", "yolo_model": "yolo11n.pt", "timestamp": "2026-06-10T09:15:22.441Z", "camera_preset": "auto", "thresholds": { "yaw_degrees": 25.0, "pitch_degrees": 20.0,

"temporal_frames": 2, "temporal_window_seconds": 3 }, "thread_safety": { "mediapipe": "thread-local", "yolo": "thread-local (one model per frame worker)", "vad": "thread-local", "audio_executor_workers": 1, "frame_executor_workers": 6, "audio_lock": "enabled", "background_dispatch": "run_in_executor (futures tracked)" } }

A.3 POST /api/debug/analyze-frame — Debug Pose Analysis

The debug endpoint returns detailed head pose information for a single frame without persisting any events to the database. It is intended for frontend calibration and candidate camera setup verification.

Sample Response:

{ "num_faces": 1, "image_size": { "width": 640, "height": 480 }, "pose": { "yaw": -8.32, "pitch": 3.15, "roll": 1.22 }, "flags": ["ALL_GOOD"], "recommendations": ["Head pose is acceptable"], "interpretation": { "yaw_meaning": "Left/Right head turn", "pitch_meaning": "Up/Down head tilt", "roll_meaning": "Head rotation (tilted ear to shoulder)", "yaw_status": "NORMAL", "pitch_status": "NORMAL" } }

A.4 GET /api/sessions/{id}/status — Session Status

The status endpoint is polled by the exam administrator dashboard to monitor the current state of an active session.

{ "session_id": "a7f3c291-84bb-4e12-9b3d-7e0c6f1d2a44", "status": "ACTIVE", "risk_score": 28.4, "events_count": 12, "elapsed_time_minutes": 23.5, "flagged_events": [{ "event_type": "MOBILE_PHONE", "severity": "CRITICAL", "timestamp": "2026-04-15T10:47:23.881Z", "metadata_": { "bbox": [142.3, 87.1, 298.7, 209.4], "model": "YOLOv11-Enhanced", "streak_frames": 3 } }] }

A.5 GET /api/sessions/{id}/report — Final Session Report

The session report endpoint is called after the exam ends to retrieve the complete audit trail and risk assessment.

{ "session_id": "a7f3c291-84bb-4e12-9b3d-7e0c6f1d2a44", "candidate_id": "CAND-2026-AV001", "exam_id": "CS301-ENDSEM-2026", "status": "FLAGGED", "overall_risk_score": 67.3, "total_events": 28, "events_by_type": { "MOBILE_PHONE": 4, "LOOKING_AWAY": 9, "BACKGROUND_SPEECH": 6, "MULTIPLE_SPEAKERS": 3, "NO_FACE": 2, "WHISPERING": 2, "LOUD_NOISE": 1, "BACKGROUND_MEDIA": 1 }, "events_by_severity": { "CRITICAL": 7, "HIGH": 12, "MEDIUM": 9 }, "duration_minutes": 127.3, "summary": "High risk. Multiple violations detected. Manual review recommended." }

Annexure B: Database Schema and Records

B.1 SessionDB Table Schema

The sessions table stores session-level information for each examination session:

Table B.1: SessionDB Table Schema

Column	Type	Constraints	Description
id	VARCHAR (UUID)	PRIMARY KEY	Unique session identifier (UUID4)
candidate_id	VARCHAR	NOT NULL	External candidate identifier from client system
exam_id	VARCHAR	NOT NULL	External exam identifier from client system
status	VARCHAR	DEFAULT 'ACTIVE'	ACTIVE / COMPLETED / FLAGGED
risk_score	FLOAT	DEFAULT 0.0	Final normalized risk score (0–100)
started_at	TIMESTAMP	DEFAULT utcnow()	Session creation timestamp (UTC)
ended_at	TIMESTAMP	NULLABLE	Session end timestamp (UTC); NULL if still active

exam_duration_minutes	INTEGER	DEFAULT 120	Planned exam duration in minutes
metadata_	JSON	DEFAULT {}	candidate_name, candidate_email, and other optional fields

B.2 EventDB Table Schema

The events table stores individual detection events generated during each session:

Table B.2: EventDB Table Schema

Column	Type	Constraints	Description
id	VARCHAR (UUID)	PRIMARY KEY	Unique event identifier (UUID4)
session_id	VARCHAR	NOT NULL, INDEX	Foreign key to sessions.id; indexed for fast per-session queries
event_type	VARCHAR	NOT NULL	MOBILE_PHONE / NO_FACE / LOOKING_AWAY / WHISPERING etc.
severity	VARCHAR	DEFAULT 'MEDIUM'	LOW / MEDIUM / HIGH / CRITICAL
confidence_score	FLOAT	DEFAULT 0.0	Model confidence 0.0–1.0
model_version	VARCHAR	DEFAULT 'Phase6'	Version tag identifying which model/phase generated this event
timestamp	TIMESTAMP	DEFAULT utcnow()	Event detection timestamp (UTC)
metadata_	JSON	DEFAULT {}	Event-specific data: bbox, dB level, pose angles, spectral features

B.3 Sample Database Records

Sample EventDB records from a test session showing different event types and their metadata structures:

Event 1 — MOBILE_PHONE:

{ "id": "e1a2b3c4-...", "session_id": "a7f3c291-...", "event_type": "MOBILE_PHONE", "severity": "CRITICAL", "confidence_score": 0.87, "model_version": "Phase6-ThreadSafe", "timestamp": "2026-04-15T10:47:23.881Z", "metadata_": { "bbox": [142.3, 87.1, 298.7, 209.4], "model": "YOLOv11-Enhanced", "source": "enhanced_object_detection", "at_edge": false, "detection_context": "center", "streak_frames": 3, "size_metadata": { "width_ratio": 0.247, "area_ratio": 0.089 }, "detection_mode": "STRICT" } }

Event 2 — LOOKING_AWAY:

{ "id": "f2b3c4d5-...", "event_type": "LOOKING_AWAY", "severity": "MEDIUM", "confidence_score": 0.94, "metadata_": { "yaw": 28.41, "pitch": -3.22, "raw_yaw": 31.05, "raw_pitch": -4.18, "smoothed": true, "streak_frames": 2, "thresholds": { "yaw": 25.0, "pitch": 20.0 }, "camera_metadata": { "focal_length": 694.2, "normalized_face_width": 0.218, "camera_preset": "auto" } } }

Event 3 — WHISPERING:

{ "id": "g3c4d5e6-...", "event_type": "WHISPERING", "severity": "HIGH", "confidence_score": 0.72, "metadata_": { "duration_seconds": 3.45, "db_level": -38.7, "spectral_flatness": 0.412, "is_low_volume": true, "is_high_flatness": true } }

Annexure C: Sample Session Report

The following is a complete sample proctoring session report generated by the GET /api/sessions/{id}/report endpoint for a simulated 30-minute test session. This report is also saved as a JSON file to the ./report/ directory on the server.

SESSION REPORT — AI PROCTORING SYSTEM (Phase 6)

==========

Session ID: a7f3c291-84bb-4e12-9b3d-7e0c6f1d2a44

Candidate: CAND-2026-AV001

Exam: CS301-ENDSEM-2026

Status: FLAGGED

Risk Score: 67.30 / 100

Duration: 29.75 minutes

Total Events: 28

Events by Type:

MOBILE_PHONE: 4 (Risk contribution: 4 × 20 × avg_conf 0.83 = 66.4 pts)

LOOKING_AWAY: 9 (Risk contribution: 9 × 3 × avg_conf 0.94 = 25.4 pts)

BACKGROUND_SPEECH: 6 (Risk contribution: 6 × 5 × avg_conf 0.85 = 25.5 pts)

MULTIPLE_SPEAKERS: 3 (Risk contribution: 3 × 18 × avg_conf 0.78 = 42.1 pts)

NO_FACE: 2 (Risk contribution: 2 × 8 × avg_conf 0.95 = 15.2 pts)

WHISPERING: 2 (Risk contribution: 2 × 12 × avg_conf 0.72 = 17.3 pts)

LOUD_NOISE: 1 (Risk contribution: 1 × 10 × avg_conf 0.88 = 8.8 pts)

BACKGROUND_MEDIA: 1 (Risk contribution: 1 × 8 × avg_conf 0.71 = 5.7 pts)

Events by Severity:

CRITICAL: 7 | HIGH: 12 | MEDIUM: 9

Summary: High risk. Multiple violations detected. Manual review recommended.

Annexure D: System Configuration

D.1 Environment Variables

The system is configured entirely via environment variables, enabling the same codebase to run in development, staging, and production environments without code changes:

Table D.1: Environment Variables Configuration

Variable	Default Value	Description
DATABASE_URL	postgresql://...neon.tech/...	Full PostgreSQL connection string with SSL parameters
REDIS_URL	redis://localhost:6379	Redis connection URL; supports redis:// and rediss:// (TLS)
CAMERA_PRESET	auto	Active camera calibration preset: auto / standard_webcam / wide_angle / phone_front / phone_wide
YOLO_MODEL_PATH	yolo11n.pt	Path to YOLO model weights file; can be overridden to use custom fine-tuned models

D.2 Pose Threshold Configuration

Pose thresholds can be adjusted at runtime via POST /api/config/thresholds without server restart:

Table D.2: Pose Threshold Configuration Parameters

Parameter	Default	Range	Description
yaw_degrees	25.0°	10–60°	Maximum allowed yaw (left-right head turn) before LOOKING_AWAY event
pitch_degrees	20.0°	10–45°	Maximum allowed pitch (up-down head tilt) before LOOKING_AWAY event
temporal_frames	2	1–5	Number of consecutive violation frames required before generating event

D.3 Required System Dependencies

The following system-level dependencies must be installed on the server before the Python application is started:

- Python 3.11+ (core runtime)
- FFmpeg 5.0+ (required by torchaudio for WebM/Opus audio decoding — installed via apt-get install ffmpeg on Ubuntu/Debian)
- libGL.so.1 (required by OpenCV — installed via apt-get install libgl1-mesa-glx)
- Redis Server 7.0+ (can be local or cloud-hosted)
- PostgreSQL client libraries (libpq — installed via apt-get install libpq-dev, required by psycopg2)

D.4 Python Package Requirements

Key Python packages and their versions used in the project:

Table D.4: Python Package Requirements

Package	Version	Purpose
fastapi	0.111+	Web framework and ASGI application
uvicorn[standard]	0.29+	ASGI server with WebSocket support
pydantic	2.7+	Data validation and settings management
mediapipe	0.10+	Face detection and facial landmark estimation
ultralytics	8.2+	YOLOv11 object detection (yolo11n.pt)
torch	2.3+	PyTorch deep learning runtime (CPU build)
torchaudio	2.3+	Audio decoding, resampling, feature extraction
librosa	0.10+	Audio analysis: MFCC, spectral features, pitch
scipy	1.13+	Welch PSD for background media detection

opencv-python	4.9+	Image processing, solvePnP, Rodrigues
sqlalchemy	2.0+	ORM and connection pooling
psycopg2-binary	2.9+	PostgreSQL driver for SQLAlchemy
redis	5.0+	Redis client library
numpy	1.26+	Numerical operations throughout all modules

CHAPTER 7: ADDITIONAL ANNEXURES

Annexure E: Internship Completion and Weekly Log

E.1 Internship Details Summary

Table E.1: Internship Details Summary

Field	Details
Student Name	Ashit Vijay
Registration Number	22BCON1528
Program	B.Tech — Computer Science and Engineering
University	JECRC University, Jaipur
Internship Organization	Algorizz Technologies Private Limited
Organization Address	1908 Tower 4, Shriram Chirpingwoods, Bengaluru 560102
Duration	January 1, 2026 – June 30, 2026 (6 months)
Role / Designation	AI/ML Developer Intern
Industry Guide	Mr. Rajneesh Mittal, Founder & CEO
Faculty Guide	Mr. Mahesh Joshi, Assistant Professor, CSE
Primary Project	AI-Powered Proctoring System for Online Examination Integrity
Technologies Used	Python, FastAPI, MediaPipe, YOLOv11, Silero VAD, librosa, torchaudio, PostgreSQL, Redis, SQLAlchemy, OpenCV, scipy

E.2 Monthly Work Log Summary

Table E.2: Monthly Work Log Summary

Month	Week	Activities Performed

January	1–2	Project kick-off meeting with Mr. Rajneesh Mittal. Requirements gathering for the AI Proctoring System. Technology stack selection research (FastAPI vs Flask vs Django; PostgreSQL vs MySQL; Redis vs Memcached). Set up development environment on Ubuntu 22.04.
January	3–4	Implemented FastAPI project scaffold with router structure. Designed and implemented PostgreSQL database schema (SessionDB and EventDB tables). Configured SQLAlchemy with QueuePool. Implemented Redis connection and basic session state caching. Built POST /sessions/create and POST /sessions/{id}/end endpoints. Deployed basic health check endpoint.
February	1–2	Researched MediaPipe Face Detection vs Haar Cascade vs DNN-based detectors. Selected MediaPipe for its production-quality pre-trained models and sub-30ms CPU inference. Integrated FaceDetection model (model_selection=1). Implemented base64 JPEG/PNG decoding pipeline using OpenCV. Discovered and applied cv2.flip() correction for unmirrored browser stream.
February	3–4	Implemented NO_FACE and MULTIPLE_FACES event generation and database persistence. Built POST /api/sessions/{id}/frames endpoint. Tested with 200 annotated frames at various face counts, distances, and lighting conditions. Tuned min_detection_confidence from 0.5 to 0.7 based on false positive analysis.
March	1–2	Researched 3D head pose estimation approaches. Selected solvePnP over simpler geometric methods for higher accuracy. Integrated MediaPipe FaceMesh with refine_landmarks=True. Implemented 6-point landmark extraction and 3D model coordinate definition. Built initial PnP solver with fixed focal length.
March	3–4	Implemented adaptive focal length estimation based on normalized inter-ocular distance. Designed and implemented 5-camera-preset configuration system. Added yaw-flip correction and off-center optical center compensation. Implemented Redis-backed 3-frame weighted average pose smoothing. Built temporal streak counter for LOOKING_AWAY event generation. Implemented PnP fallback using geometric landmarks.
April	1–2	Researched audio analysis libraries: torchaudio, librosa, pyaudio, soundfile. Selected torchaudio for decoding (FFmpeg backend) and librosa for feature extraction. Integrated Silero VAD from PyTorch Hub. Implemented 4-

		attempt audio format fallback pipeline. Implemented BACKGROUND_SPEECH detector. Implemented MULTIPLE_SPEAKERS detector (initial threshold: mfcc_std>3.0 OR pitch_std>40).
April	3–4	Implemented WHISPERING detector (initial — incorrect spectral flatness direction). Implemented SPEECH_PATTERN detector (reading and Q&A rhythm). Implemented LOUD_NOISE detector. Implemented BACKGROUND_MEDIA detector (Welch PSD). Built POST /api/sessions/{id}/audio endpoint. Tested all 6 audio detectors with synthetic test audio files.
May	1–2	Researched YOLO model variants. Selected YOLOv8n as initial choice. Integrated Ultralytics YOLO. Implemented DETECTION_CONFIG with per-class thresholds. Built is_at_frame_edge() and is_valid_partial_detection() helper functions. Implemented EnhancedObjectDetector class. Applied torch.manual_seed(42) for deterministic inference.
May	3–4	Implemented Redis temporal IoU tracking (5-frame history, calculate_iou helper). Identified malloc heap corruption crash from log analysis. Diagnosed background_tasks.add_task(executor.submit) anti-pattern as root cause (FIX 1). Fixed by switching to loop.run_in_executor(). Implemented threading.local() for MediaPipe (FIX 2) and YOLO (FIX 3). Upgraded YOLOv8n to YOLOv11n (FIX 4).
June	1–2	Corrected whispering detection spectral flatness direction (FIX 5). Added -50 dBFS energy gate to background media detector (FIX 6). Conducted concurrency stress test (500 concurrent audio request pairs — zero crashes post-fix). Performed false positive analysis across all 6 audio detectors. Tuned MULTIPLE_SPEAKERS to mfcc_std>6.0 AND pitch_std>80 Hz. Tuned BACKGROUND_SPEECH to >3.5s and >65% ratio.
June	3–4	Built and tested /api/debug/analyze-frame endpoint for frontend calibration. Implemented GET /api/sessions/{id}/report with JSON file persistence. Finalized risk scoring weights for all 16 event types. Conducted 50-session end-to-end integration test. Deployed to Neon PostgreSQL cloud instance. Configured deployment documentation. Prepared end-semester internship report.

Annexure F: Technical Glossary

The following technical terms are used throughout this report:

Table G.1: Glossary of Technical Terms

Term	Definition
ASGI	Asynchronous Server Gateway Interface — the Python web server interface standard for async web frameworks (FastAPI, Starlette, Quart).
MFCC	Mel-Frequency Cepstral Coefficient — a feature representation of the short-term power spectrum of audio, widely used in speech and speaker recognition.
VAD	Voice Activity Detection — automatic detection of the presence or absence of speech in an audio signal.
PnP	Perspective-n-Point — the problem of estimating the pose (rotation and translation) of an object from n 3D-2D point correspondences.
solvePnP	OpenCV function that solves the PnP problem given 3D model points and their 2D image projections.
IoU	Intersection over Union — the ratio of the intersection area to the union area of two bounding boxes. Used to measure detection overlap between frames.
PSD	Power Spectral Density — a measure of signal's power distribution as a function of frequency.
Welch's Method	A method for estimating PSD by averaging periodograms of overlapping windowed signal segments, reducing estimation variance.
Spectral Flatness	The ratio of the geometric mean to the arithmetic mean of a signal's power spectrum. Values near 0 = tonal (harmonic), values near 1 = noise-like.
Thread Safety	The property of code that functions correctly when accessed from multiple threads simultaneously without requiring external synchronization.

threading.local()	Python's built-in mechanism for thread-local storage — each thread gets its own independent copy of the stored data.
GIL	Global Interpreter Lock — Python's mutex that prevents multiple native threads from executing Python bytecodes simultaneously. Does NOT protect C extension code.
ThreadPoolExecutor	Python's concurrent.futures class that manages a pool of worker threads and queues submitted callables for execution.
run_in_executor	asyncio method to dispatch blocking work from the async event loop to a thread pool, returning a Future.
malloc	Memory allocation function in the C standard library. 'malloc heap corruption' indicates concurrent threads have corrupted the heap's free list.
QueuePool	SQLAlchemy's default connection pool — maintains a queue of idle connections and returns them to callers, creating new ones up to max_overflow when the pool is exhausted.
TTL	Time To Live — the duration after which a Redis key is automatically deleted. Used to implement time-bounded state (pose history, object streaks).
CAGR	Compound Annual Growth Rate — the rate at which something (e.g., a market) grows on a compounded basis year over year.
ORM	Object-Relational Mapper — software that converts between object-oriented programming objects and relational database tables (e.g., SQLAlchemy).
YOLO	You Only Look Once — a family of real-time object detection neural network architectures that detect objects in a single forward pass.
dBFS	Decibels relative to Full Scale — a logarithmic unit for measuring audio amplitude. 0 dBFS is the maximum possible digital signal level; all values are negative.

Annexure G: Project Repository Structure

The AI Proctoring System codebase is organized as a single-file FastAPI application with supporting directories for reports and debug audio. The directory structure is as follows:

proctoring-system/

```
├── main.py                # Complete FastAPI application (Phase 6)
├── yolo11n.pt             # YOLOv11 nano model weights
├── requirements.txt       # Python package dependencies
├── .env                   # Environment variables (not committed to VCS)
├── .env.example           # Template showing required env vars
├── README.md              # Project setup, deployment, and API documentation
├── report/                # Auto-generated session JSON reports
│   └── report_{session_id}.json
├── debug_audio/           # Auto-saved audio debug WAV files
│   └── {session_id}_{timestamp}.wav
└── tests/                 # Unit and integration test files
    ├── test_face_detection.py
    ├── test_head_pose.py
    ├── test_yolo_detection.py
    ├── test_vad.py
    ├── test_audio_detectors.py
    ├── test_concurrency.py
    └── test_integration.py
```

Annexure H: Comparison of Proctoring Approaches

The following table compares the AI Proctoring System developed during this internship against leading commercial alternatives and alternative architectural approaches:

Table 4.3: System Comparison Against Commercial Alternatives

Dimension	This System (Algorizz)	ProctorU / Examity	Honorlock
Architecture	Open-source API backend; self-hostable	Proprietary SaaS; cloud-only	Browser extension + cloud
Cost Model	Infrastructure cost only; no per-exam fee	USD 5–30 per candidate exam	~USD 6.50 per exam hour
Video Detection	MediaPipe + YOLOv11 (CPU)	Proprietary CV models	AI + human review hybrid
Audio Detection	6 independent detectors (VAD, speakers, whisper, patterns, noise, media)	Basic voice detection	Limited audio monitoring
False Positive Rate	< 5% after Phase 6 tuning	Not publicly disclosed	Reported as ~15% by critics
Transparency	Full source code; all thresholds configurable	Black box	Partially disclosed
Latency	200–500ms background processing; <10ms API response	Variable (cloud-dependent)	Variable (extension-based)
GPU Required	No (CPU-only, suitable for commodity hardware)	Yes (cloud GPUs)	Yes (cloud GPUs)
Data Privacy	All data stays on self-hosted server	Data sent to US cloud servers	Data sent to US cloud servers
LMS Integration	REST API (custom integration needed)	Native LMS plugins available	Canvas, Blackboard, Moodle

The primary differentiating advantage of the Algorizz proctoring system is its open-architecture, self-hostable design combined with a low per-session marginal cost. For educational institutions in India and other cost-sensitive markets that conduct large-scale

examinations (thousands of candidates), the elimination of per-exam fees represents a substantial cost reduction compared to commercial alternatives.

Annexure I: Learning Reflection Journal

The following is a reflective journal maintained throughout the internship, documenting key learning moments and observations:

January 2026 — Week 3 Reflection

"Setting up the database schema was more complex than I expected. Designing the metadata_ JSON column for EventDB was a key insight — rather than creating separate columns for every possible event attribute (bounding box coordinates, dB levels, spectral features), using a single JSONB column allows each event type to store whatever metadata is relevant without requiring schema migrations when new event types are added. This is a pattern I had read about in documentation but never applied in a real project. Seeing how it simplifies the code (one EventDB class handles 16 event types) was a concrete lesson in schema design for heterogeneous data."

March 2026 — Week 2 Reflection

"The head pose estimation problem was the most mathematically challenging part of the project. I spent two days reading about the PnP problem, camera intrinsics, and Euler angle extraction before writing a single line of code. The key insight was that without knowing the camera focal length, the system cannot accurately recover the absolute yaw and pitch angles — the same face displacement in pixels can result from a small head turn close to the camera or a large head turn far from the camera. The adaptive focal length estimation was an elegant engineering solution: using the face size itself as a proxy for camera distance and FOV. When it worked correctly on the first real test — producing yaw angles within 5° of the known ground truth — it was one of the most satisfying moments of the project."

May 2026 — Week 4 Reflection (Phase 6 malloc crash)

"The malloc crash was simultaneously the most frustrating and most educational experience of the internship. When the system first started crashing in production, I had no idea where to start — malloc heap corruption is not a Python error with a traceback pointing to a specific line. I had to learn to read the application logs as a narrative: what happened just before the crash? The two 'process_audio CALLED' entries with identical audio lengths was the key clue. It proved that two audio chunks were being processed simultaneously. From there, understanding why they were simultaneous despite the audio_executor required reading the FastAPI documentation carefully and understanding that background_tasks.add_task() does not behave the way I assumed. The lesson: never assume how a framework handles concurrency — always verify by reading the source code or official documentation."

June 2026 — Final Week Reflection

"Six months feels both very long and very short. Very long because of the depth of learning — I went from never having written a production API to building a system that handles concurrent multi-modal AI processing. Very short because there is so much more I wanted to build: the WebSocket real-time dashboard, the speaker diarization upgrade, the Docker containerization. The experience of having unfinished goals at the end of a project was itself a professional lesson: real engineering projects are never fully done; they are versions, each making the system more capable, more reliable, and more useful. I leave this internship not just with a completed system but with a clear understanding of what the next version should be and why — which is exactly the right state to be in."

REFERENCES AND BIBLIOGRAPHY

References are formatted using the Harvard System of Referencing as prescribed by JECRC University guidelines.

Books

[1] Bradski, G. and Kaehler, A. (2019) Learning OpenCV 4: Computer Vision in C++ with the OpenCV Library. 3rd edn. Sebastopol, CA: O'Reilly Media.

[2] Géron, A. (2022) Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow. 3rd edn. Sebastopol, CA: O'Reilly Media.

[3] Ramalho, L. (2022) Fluent Python: Clear, Concise, and Effective Programming. 2nd edn. Sebastopol, CA: O'Reilly Media.

[4] Percival, H. and Gregory, B. (2020) Architecture Patterns with Python: Enabling Test-Driven Development, Domain-Driven Design, and Event-Driven Microservices. Sebastopol, CA: O'Reilly Media.

[5] Owens, M. and Allen, G. (2022) SQLite. 3rd edn. Sebastopol, CA: O'Reilly Media. [Note: SQLAlchemy design principles referenced.]

Journal Articles and Conference Papers

[6] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.L., Yong, M.G., Lee, J. and Chang-Hyun, W. (2019) 'MediaPipe: A Framework for Building Perception Pipelines', arXiv preprint arXiv:1906.08172.

[7] Redmon, J. and Farhadi, A. (2018) 'YOLOv3: An Incremental Improvement', arXiv preprint arXiv:1804.02767.

[8] McFee, B., Raffel, C., Liang, D., Ellis, D.P., McVicar, M., Battenberg, E. and Nieto, O. (2015) 'librosa: Audio and Music Signal Analysis in Python', Proceedings of the 14th Python in Science Conference, Austin, TX, pp. 18-25.

[9] Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J. and van der Walt, S.J. (2020) 'SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python', Nature Methods, 17, pp. 261-272.

[10] Snyder, B. and Bregman, M. (2020) 'Automatic Cheating Detection in Online Examination Systems: A Comprehensive Survey', International Journal of Advanced Research in Computer Science, 11(3), pp. 45-58.

[11] Nigam, A., Pasricha, R., Singh, T. and Churi, P. (2021) 'A Systematic Review on AI-based Proctoring Systems: Past, Present and Future', Education and Information Technologies, 26(5), pp. 6421-6445.

Online Resources

[12] FastAPI (2024) FastAPI Documentation. Available at: https://fastapi.tiangolo.com (Accessed: 15 June 2026).

[13] Google AI (2024) MediaPipe Solutions Guide. Available at: https://ai.google.dev/edge/mediapipe/solutions/guide (Accessed: 10 June 2026).

[14] Ultralytics Inc. (2024) YOLOv11 Documentation. Available at: https://docs.ultralytics.com (Accessed: 12 June 2026).

[15] Silero Team (2023) Silero VAD: Pre-trained Enterprise-Grade Voice Activity Detector. Available at: https://github.com/snakers4/silero-vad (Accessed: 5 March 2026).

[16] PyTorch Team (2024) torchaudio Documentation. Available at: https://pytorch.org/audio/stable/index.html (Accessed: 8 June 2026).

[17] SQLAlchemy Authors (2024) SQLAlchemy 2.0 ORM Documentation. Available at: https://docs.sqlalchemy.org (Accessed: 20 May 2026).

[18] Redis Ltd. (2024) Redis 7.x Documentation. Available at: https://redis.io/docs (Accessed: 22 May 2026).

[19] Neon Technologies (2024) Neon Serverless Postgres Documentation. Available at: https://neon.tech/docs (Accessed: 1 April 2026).

[20] Algorizz Technologies (2024) Algorizz: AI, Data and Digital Transformation Solutions. Available at: https://algorizz.com (Accessed: 15 January 2026).

[21] Python Software Foundation (2024) threading — Thread-based parallelism. Python 3.11 Documentation. Available at: https://docs.python.org/3/library/threading.html (Accessed: 3 May 2026).

[22] Python Software Foundation (2024) asyncio — Asynchronous I/O. Python 3.11 Documentation. Available at: https://docs.python.org/3/library/asyncio.html (Accessed: 3 May 2026).

[23] OpenCV Team (2024) OpenCV: Camera Calibration and 3D Reconstruction (solvePnP). Available at: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html (Accessed: 10 March 2026).

[24] Grand View Research (2024) Online Proctoring Market Size, Share & Trends Analysis Report. Available at: https://www.grandviewresearch.com/industry-analysis/online-proctoring-market-report (Accessed: 18 January 2026).

[25] NumPy Developers (2024) NumPy 1.26 Reference. Available at: https://numpy.org/doc/stable/reference (Accessed: 15 June 2026).